Efficient Federated Learning With Mean Block Difference-Based Global Aggregation and Patience-Based Local Training

Chuan Sun, *Member, IEEE*, Yuanyuan Chen, *Member, IEEE*, Xiaoli Tang, and Han Yu, *Senior Member, IEEE*

***Abstract*—As neural network models grow larger and more complex, federated learning (FL) faces challenges in terms of communication and computation efficiency. To address these issues, layer-wise learning has been proposed. Existing approaches did not leverage useful properties of layer-wise learning including update locking and variations in convergence rates, thereby resulting in sub-par model performance. To bridge this gap, we propose the Federated Mean Block Difference-based global model aggregation approach with Patience-based local training (`FedMBDP`). We automatically partition the neural network model into uncoupled blocks and progressively train them. Determining which blocks to train and aggregate becomes a critical task. To improve computation efficiency, we propose a patience-based local training algorithm to adaptively select training blocks, reducing computation latency. To improve communication efficiency, we introduce a mean block difference-based global model aggregation algorithm to dynamically select blocks for aggregation to minimize communication latency. We provide the convergence analysis of `FedMBDP`. Extensive experiments on three widely adopted benchmark datasets show that `FedMBDP` achieves superior performance compared to six state-of-the-art approaches. It reduces FL training latency by 26.37% compared to the best baseline, while achieving similar test accuracy.**

***Index Terms*—Federated learning (FL), communication and computation efficiency, layer-wise learning.**

I. Introduction

FEDERATED learning (FL) has emerged as a promising paradigm that preserves data privacy while leveraging distributed computing resources for model training [1], [2], [3]. By enabling multiple parties to collaboratively train a shared model without exposing their local data, FL embodies the principles of privacy preservation [4], [5], [6]. Each participant, often referred to as a "client," trains the model locally on its dataset and only shares model updates with a central server, thus minimizing the risk of data leakage. Consequently, FL has found widespread applications across various domains, including finance, healthcare, and edge computing [7], [8].

Recent advances in deep neural networks, including deep convolutional networks and large language models, have demonstrated that increasing model depth and width can substantially improve performance in many tasks [9]. However, such large models, often containing millions or even billions of parameters, require vast computational resources and energy for training [10], raising concerns about scalability. With the rapid increase in model size, the communication and computation burden in FL becomes increasingly severe, especially when models are deployed on resource-constrained edge devices [11], [12]. In particular, transmitting large-scale model updates over heterogeneous networks can easily saturate bandwidth and increase training latency, which motivates the use of efficiency-oriented techniques such as quantization and pruning to reduce update size while preserving accuracy.

In the context of FL, the convergence of burgeoning model sizes and resource-intensive computations exacerbates the inherent challenges associated with communication bottlenecks and computational inefficiencies [13]. As each participating device engages in iterative model updates and parameter exchanges, the overhead of transmitting and aggregating vast amounts of data across heterogeneous networks becomes a significant limiting factor. These devices, often operating with varying computational capacities and network conditions, contribute to asynchronous updates, further complicating the coordination of the global model [14]. The communication latency introduced by these factors is particularly concerning in latency-sensitive edge applications, where the training process must be completed within a strict time budget. Consequently, the potential of FL to unlock the collective intelligence of distributed datasets is hindered by practical constraints such as communication bandwidth, energy consumption, and computational capacity.

We make two key observations. *i)* During the training of deep neural networks, each layer must wait for the signal to propagate through the entire network before being updated (a phenomenon known as the update locking problem) [15]. This introduces significant delays, particularly in deep architectures where gradients take longer to flow through the network, resulting in

Received 14 October 2024; revised 2 March 2026; accepted 6 April 2026. Date of publication 15 April 2026; date of current version 2 June 2026. This work was supported in part by Chongqing Municipal Social Science Planning Project under Grant 2025BS055, in part by the Ministry of Education, Singapore, under its Academic Research Fund Tier 1 under Grant RG101/24, in part by RIE2025 Industry Alignment Fund Industry Collaboration Projects (IAF-ICP) under Award I2301E0026, administered by A∗STAR, in part by the Chongqing Normal University Foundation Projection for Doctoral Start-up Foundation under Grant 25XLB026, and in part by Alibaba Group and NTU Singapore through Alibaba-NTU Global e-Sustainability CorpLab (ANGEL). Recommended for acceptance by Y. Gao. *(Corresponding author: Han Yu.)*

Chuan Sun is with the School of Computer and Information Science, Chongqing Normal University, Chongqin 401331, China (e-mail: sunchuan0602@gmail.com).

Yuanyuan Chen, Xiaoli Tang, and Han Yu are with the College of Computing and Data Science (CCDS), Nanyang Technological University (NTU), Singapore 639798 (e-mail: yuanyuan003@e.ntu.edu.sg; xiaoli001@e.ntu.edu.sg; han.yu@ntu.edu.sg).

Digital Object Identifier 10.1109/TKDE.2026.3684304

1041-4347 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on October 27,2026 at 23:55:19 UTC from IEEE Xplore. Restrictions apply.

inefficient training. To address this, layer-wise learning has been proposed as a solution. This approach allows for independent or partially independent training of each layer, thus decoupling updates and improving training efficiency [16], [17], [18]. *ii)* Additionally, previous studies have shown that the shallower layers of deep neural networks tend to converge faster than the deeper layers [19], [20]. This disparity in convergence speed can be attributed to the flatter local minima typically found in the loss landscapes of shallower layers, which provide more stable and predictive gradients. As a result, the training process for these layers is not only faster but also more resilient to noisy updates, offering opportunities for optimization in FL settings with limited communication and computational resources [20].

Motivated by the above insight, we propose a Federated Mean Block Difference-based global model aggregation approach with Patience-based local training (`FedMBDP`) to reduce the training latency of FL. Considering the interdependence between network layers, `FedMBDP` begins by automatically grouping related layers into blocks and adopts a block-wise learning approach. Then, it decomposes the training efficiency in FL into two key components: *i)* local training, and *ii)* global model aggregation. For local training, `FedMBDP` optimizes three critical decisions: a) when to train deeper blocks, b) when to freeze shallower blocks, and c) how to balance computation latency differences due to heterogeneity across devices. To this end, we propose a patience-based algorithm that adaptively determines the appropriate timing for those actions, optimizing the local training processing. Regarding global model aggregation, we focus on identifying which blocks shall be aggregated and propose a mean block difference-based algorithm that dynamically selects the blocks to be aggregated, minimizing communication latency while ensuring effective model updates. These strategies collectively optimize block-wise local computation and selective communication to improve the training-time efficiency of FL for deep models.

We conduct extensive experiments comparing 6 state-of-the-art baselines across three widely adopted benchmark datasets using three neural network architectures. The results demonstrate that `FedMBDP` is capable of significantly reducing training latency by 26.37% compared to the best baseline, while maintaining good model performance.

The rest of this paper is organized as follows. Section II reviews related work. Section III models the optimization problem and the proposed approach is given in Section IV. The convergence analysis is given in Section V. Numerous experiments are conducted in Section VI. Section VII concludes this paper.

II. Related Work

A. Communication Efficiency in FL

Recently, various methods [26], [27], [28] have been proposed to improve communication efficiency in FL. For instance, FetchSGD [29] introduced sketching to summarize local updates, reducing communication costs by transmitting only essential information. Similarly, [28] explored quantizing global model updates to compress parameters into low-bit representations, mitigating communication overhead without compromising accuracy. FedPAQ [22] alleviated communication bottlenecks through periodic averaging and quantization, while Fjord [30] addressed heterogeneous targets in FL by introducing ordered dropout techniques to enhance communication efficiency. However, these approaches primarily focus on communication efficiency, overlooking the equally important aspect of computation efficiency.

B. Computation Efficiency in FL

To further reduce computation costs, several approaches have been proposed. Federated Dropout (FD) [23] leverages dropout regularization techniques to enable FL on resource-constrained devices. Similarly, HeteroFL [24] assigns the same submodel to devices regardless of their convergence status, leading to inefficient resource utilization. Hermes [31] applies structured pruning to identify sparse structures in local models before training pruned models with fixed structures. Successive Layer Training (SLT) [25] enables successive freezing and training of local models to reduce the training's resource requirements. While these techniques reduce computational demands, they may compromise neural network performance due to structural changes. Recent efforts, such as FedBFPT [13] and ProgFed [32], focus on partitioning local models and progressively training partitions to reduce computation. However, these methods still face challenges in addressing resource heterogeneity and may not fully optimize forward and backward propagation computations.

Most existing works predominantly focus on mitigating communication costs while often overlooking computational inefficiency. Although communication-efficient approaches, such as dropout and structured sparsification, reduce transfer overheads, they typically modify model architectures or lossily compress gradients, risking non-negligible accuracy degradation. Furthermore, state-of-the-art layer-wise methods [13], [25], [32] mainly reduce backpropagation costs by freezing upper layers. However, they fail to optimize forward propagation and necessitate uploading the full model parameters in each round, thereby lacking a joint optimization of computation and communication latency. In contrast, `FedMBDP` distinguishes itself by: *i)* synergistically reducing both latencies via uncoupled block-wise learning and selective aggregation; *ii)* adaptively freezing and uploading blocks based on client-specific latency constraints and real-time convergence dynamics; and *iii)* providing rigorous theoretical guarantees for the selective update mechanism.

As summarized in Table I, we provide a systematic comparison of efficiency mechanisms. We categorize existing layer-wise methods, such as SLT and FedBFPT, as achieving only 'Partial Efficiency' (○). This classification stems from their limitation to optimizing backward propagation, while leaving forward propagation costs and communication overheads largely unaddressed (indicated by −). Conversely, `FedMBDP` attains 'High Efficiency' (•) across all dimensions by holistically optimizing the training scope (via uncoupled forward/backward passes) and the transmission payload (via MBD-based selective aggregation).

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on October 27,2026 at 23:55:19 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON OF FedMBDP WITH STATE-OF-THE-ART ALGORITHMS. **EFFICIENCY RATING**: • = HIGH (JOINT/FULL OPT.), ○ = MEDIUM (PARTIAL OPT.), – = LOW (NO OPT.)

Method	Computation Optimization			Communication Optimization			Adaptive
	Mechanism	Scope	Eff.	Mechanism	Payload	Eff.	
Standard & Comm-Efficient Methods							
FedAvg [21]	None	Full / Full	–	None	Full Model	–	–
FedPAQ [22]	None	Full / Full	–	Quantization	Compressed	•	–
Computation-Efficient (Pruning/Dropout)							
FD [23]	Dropout	Partial / Partial	•	Dropout	Partial	•	–
HeteroFL [24]	Width Pruning	Partial / Partial	•	Static Pruning	Partial	•	–
Layer-wise / Progressive Methods							
SLT [25]	Sequential Freezing	Full / Partial (Backprop only)	○	None	Full Model (Upload all)	–	Fixed Schedule
FedBFPT [13]	Model Partition	Full / Partial (Backprop only)	○	None	Full Model (Upload all)	–	Heuristic Rules
FedMBDP	**Uncoupled Learning**	**Partial / Partial (Fwd & Bwd)**	•	**MBD-based**	**Selective Blocks**	•	**Patience**

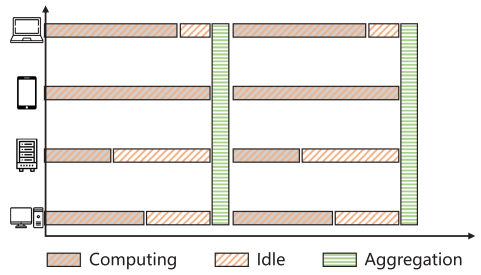


Fig. 1. Training Latency of FL under heterogeneous environments.

III. SYSTEM MODEL AND PROBLEM FORMULATION

A. Federated Learning in Heterogeneous Edge-Cloud Networks

We consider the FL in a heterogeneous edge-cloud network. The network consists of N edge servers (denoted as $\mathcal{N} = \{1, 2, \ldots, N\}$) and one cloud server (denoted as o). All edge servers are connected to the cloud server through wired optical cables, ensuring a stable and uninterrupted connection. Each edge server acts as a client, training the local model based on its respective local data. During the model training process, the cloud server periodically aggregates the local model parameters from the edge servers and broadcasts the updated global model parameters back to them. It is important to note that the edge servers possess varying computing and communication resources. These heterogeneous characteristics directly affect the efficiency of FL, which is reflected in the training latency as shown in Fig. 1.

We denote the local datasets as $\mathcal{D} = \{\mathcal{D}_1, \mathcal{D}_2, \ldots, \mathcal{D}_N\}$, where D_n represents the dataset at edge server n. These datasets are assumed to be Non-Independently and Identically Distributed (Non-IID), a scenario that can affect the convergence performance of FL. The local model parameters at edge server n are denoted as $\mathbf{w}_n$. Typically, a model comprises layers representing various neural network structural patterns like Convolution, Pooling, Normalization, Activation, and Encoder. Consequently, we can categorize a model into different blocks based on these patterns (detailed in Section IV-B). Let $\mathcal{K} = \{1, 2, \ldots, K\}$ denote the set of blocks. The local model parameters $\mathbf{w}_n$ can be further expressed as $\mathbf{w}_n = (\mathbf{w}_n^1, \mathbf{w}_n^2, \ldots, \mathbf{w}_n^K)$, where $\mathbf{w}_n^k$ represents the parameters of block k. Different neural networks may have different sets of blocks $\mathcal{K}$. In this study, we assume that all edge servers share the same neural network structure.

The cloud server maintains a global model $\mathbf{w}$, and the objective of FL is to learn the optimal $\mathbf{w}^*$ that minimizes the global loss function, as expressed below

$$\min_{\mathbf{w}} F(\mathbf{w}) := \sum_{n \in \mathcal{N}} p_n F_n(\mathbf{w}_n), \tag{1}$$

where p_n is an aggregation weight such that $p_n > 0$ and $\sum_{n \in \mathcal{N}} p_n = 1$; $F_n(\mathbf{w}_n)$ is the local loss function for edge server n, expressed as

$$F_n(\mathbf{w}_n) = \frac{1}{|\mathcal{D}_n|} \sum_{x \in \mathcal{D}_n} \ell(\mathbf{w}_n; x), \forall n \in \mathcal{N}, \tag{2}$$

where $\ell(\mathbf{w}_n; x)$ is a user-specified loss function.

FL consists of $\mathcal{I} = \{1, 2, \ldots, I\}$ global aggregation rounds. Each global aggregation round i includes $\mathcal{T}_i = \{1, 2, \ldots, T_i\}$ local training iterations. In each iteration t_i of round i, edge server n updates its local model $\mathbf{w}_n(t_i)$ following a set of rules defined by the FL algorithm:

$$\mathbf{w}_n(t_i) = \mathbf{w}_n(t_i - 1) - \alpha \nabla F_n(\mathbf{w}_n(t_i - 1)),$$
$$\forall n \in \mathcal{N}, t_i \in \mathcal{T}_i, i \in \mathcal{I}, \tag{3}$$

where α is a learning rate.

After T_i local training iterations, all edge servers will upload local model parameters to the cloud server for global aggregation. In round i, the global aggregation at the cloud server is expressed as

$$\mathbf{w}(i) = \sum_{n \in \mathcal{N}} p_n \mathbf{w}_n(i), \forall i \in \mathcal{I}. \tag{4}$$

B. Computation and Communication Model

Massive local model training and overload parameter transmission can significantly increase the training latency in federated learning. Therefore, we aim to improve the efficiency of federated learning by reducing the training latency, including computation latency and communication latency.

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on October 27,2026 at 23:55:19 UTC from IEEE Xplore. Restrictions apply.

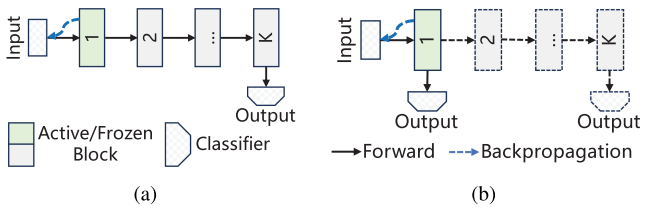


Fig. 2. Illustration of Two Learning Manners: (a) Traditional Learning; (b) Block-wise Learning.

1) Computation Latency: To address the computational limitations of edge servers, we employ a block-wise training technique, as shown in Fig. 2, to alleviate the computational demands of training local models [33]. Unlike traditional end-to-end training, which requires forward propagation through the entire network even when only a subset of blocks is being trained (Fig. 2(a)), our approach introduces local classifiers to intercept the forward signal at the end of each block. As illustrated in Fig. 2(b), when training a specific block (e.g., block 1), all computations, both forward and backward, are confined to that block and its associated auxiliary network. This design ensures the validity of our independence assumption and leads to a substantial reduction in floating-point operations (FLOPs). The difference in FLOPs between the two methods is significant. In traditional learning, even if only block 1 is being trained and later blocks are frozen, forward propagation still traverses the entire network, limiting FLOPs savings to the backward pass. In contrast, our block-wise learning method uses auxiliary networks to truncate forward propagation, so that training a given block involves computations only within that block and its auxiliary network during both forward and backward passes. This results in a measurable and considerably lower FLOPs count compared to conventional end-to-end training.

Since the layers within a block are coupled to each other, we conduct training on local models block-by-block. The computation latency is influenced by three key factors: the local data size, the local neural network size, and the computation capacity available at each edge server. The computing resources required during model training are generally quantified as the number of FLOPs [34]. Models with more blocks typically require more FLOPs during training. To reduce computation latency, it is crucial to judiciously determine which blocks should be involved in the training process.

Definition 1: The computation variable set $\mathcal{X} = \{\mathbf{x}_n(t_i)\}_{n\in\mathcal{N},t_i\in\mathcal{T}_i,i\in\mathcal{I}}$ determines which blocks participate in training, where $\mathbf{x}_n(t_i) = (x_n^1(t_i), x_n^2(t_i), \ldots, x_n^K(t_i))$ and $x_n^k(t_i) = 1$ indicates that block k will be trained; otherwise, block k will be excluded from training.

Let $\mu(\mathbf{w}_n(t_i)\mathbf{x}_n^\top(t_i))$ be the number of FLOPs required by edge server n during training iteration t_i of round i to process one data sample, and C_n represents the unit average computation latency for one FLOP at edge server n. Then, the computation latency $l_n^{cpt}(t_i)$ is calculated as

$$l_n^{\text{cpt}}(t_i) = |\mathcal{D}_n|\mu\left(\mathbf{w}_n(t_i)\mathbf{x}_n^\top(t_i)\right) C_n, \forall n \in \mathcal{N}, t_i \in \mathcal{T}_i, i \in \mathcal{I}. \tag{5}$$

2) Communication Latency: During the training process, some blocks of the neural network model may exhibit less variation and, therefore, may not be required to be uploaded to the cloud server for aggregation [11]. To reduce communication latency, we selectively upload only the important blocks that have undergone significant changes by introducing a communication variable set to determine which blocks should be uploaded for aggregation.

Definition 2: The communication variable set $\mathcal{Y} = \{\mathbf{y}_n(i)\}_{n\in\mathcal{N},i\in\mathcal{I}}$ determines which blocks to upload, where $\mathbf{y}_n(i) = (y_n^1(i), y_n^2(i), \ldots, y_n^K(i))$ and $y_n^k(i) = 1$ indicates that block k will be uploaded; otherwise, block k will not be uploaded.

Let $\beta(\mathbf{w}_n(i)\mathbf{y}_n^\top(i))$ denote the size of the model parameters $\mathbf{w}_n$ to be uploaded from edge server n in round i. As downloading is typically much quicker than uploading [21], [35], [36], we focus on the communication latency of uploading model parameters and ignore downloading. Then the communication latency $l_n^{cmt}(i)$ is calculated as

$$l_n^{\text{cmt}}(i) = \beta\left(\mathbf{w}_n(i)\mathbf{y}_n^\top(i)\right) R_n, \forall n \in \mathcal{N}, i \in \mathcal{I}, \tag{6}$$

where R_n is the mean communication latency for transmitting a single sample from the edge server n to the cloud server.

3) Training Latency: Based on (5) and (6), the training latency of round i can be expressed as shown in (7). In (7), part I represents the computation latency associated with the T_i local training iterations at edge server n in round i. Part II represents the latency when the cloud server receives the local model parameters from edge server n. Part III represents the longest waiting latency for the cloud server to receive all local models from the edge servers, as edge servers operate in parallel.

$$L(i) = \underbrace{\max_{n\in\mathcal{N}}\{\underbrace{\sum_{t_i\in\mathcal{T}_i} l_n^{\text{cpt}}(t_i)}_{\text{part I}} + \overbrace{l_n^{\text{cmt}}(i)}^{\text{part II}}\}}_{\text{part III}}, \forall i \in \mathcal{I}. \tag{7}$$

C. Problem Formulation

The overall objective of our proposed `FedMBDP` method is to minimize the total training latency across all rounds by optimizing the variables $\mathcal{X}$ and $\mathcal{Y}$, subject to the constraints imposed by the available computation and communication resources. This optimization problem can be formulated as:

$$\begin{aligned} &\min_{\mathcal{X},\mathcal{Y}} \sum_{i\in\mathcal{I}} L(i) \\ &\text{s.t. c1: } F(\mathbf{w}(I)) - F(\mathbf{w}^*) \le \epsilon, \\ &\quad \text{c2: } x_n^k(t_i) \in \{0,1\}, \forall t_i \in \mathcal{T}_i, i \in \mathcal{I}, n \in \mathcal{N}, k \in \mathcal{K}, \\ &\quad \text{c3: } y_n^k(i) \in \{0,1\}, \forall i \in \mathcal{I}, n \in \mathcal{N}, k \in \mathcal{K}, \end{aligned} \tag{8}$$

where constraint c1 is the convergence requirement, ensuring that the difference between the global loss function $F(\mathbf{w}(I))$ after I aggregation rounds and the optimal loss function $F(\mathbf{w}^*)$ is within a specified convergence threshold $\epsilon > 0$, which is close

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on October 27,2026 at 23:55:19 UTC from IEEE Xplore. Restrictions apply.

to zero. The constraints c2 and c3 are used to ensure that the computation and communication variables are binary.

Eq. (8) does not directly optimize the model parameters; instead, it determines the block-wise computation and communication decisions. Therefore, our approach aims to achieve efficient FL by minimizing the total training latency while maintaining robust model performance, as specified by the convergence requirement.

IV. The Proposed FedMBDP Approach

In this section, we first analyze and decompose the formulated problem into two subproblems. Then, we present a FedMBDP approach to solve this optimization problem. Finally, we give a detailed analysis of computational complexity.

A. Problem Analysis and Decomposition

Given that all variables in Eq. (8) are integers, the optimization problem falls under the category of Integer Linear Programming (ILP), a well-known NP-hard problem [37], [38]. The intrinsic complexity of ILP makes obtaining an exact optimal solution computationally prohibitive, particularly for large-scale instances. The challenge is further compounded by the intricate spatial and temporal interdependencies between decisions related to computation and communication, which amplify the difficulty of solving this problem efficiently. Specifically, identifying an optimal solution necessitates an exhaustive search through the vast combinatorial space of possible integer assignments for the decision variables. As the problem scale and the number of variables increase, the computational demands escalate significantly. To tackle this, we describe how FedMBDP approximately solves the latency-aware problem (8) in each round. We focus on a particular case within a single iteration $i \in \mathcal{I}$, allowing us to gain analytical insights into the complexity. This approach enables a more manageable decomposition of the original problem, which we can reformulate as follows:

$$\min_{\substack{\{\mathbf{x}_n(t_i)\}_{n\in\mathcal{N},t_i\in\mathcal{T}_i},\\ \{\mathbf{y}_n(i)\}_{n\in\mathcal{N}}}} \left\{ \max_{n\in\mathcal{N}} \left\{ \sum_{t_i\in\mathcal{T}_i} |\mathcal{D}_n|\mu(\mathbf{w}_n(t_i)\mathbf{x}_n^\top(t_i))C_n + \beta(\mathbf{w}_n(i)\mathbf{y}_n^\top(i))R_n \right\} \right\}$$

$$\text{s.t. c2 and c3.} \tag{9}$$

Eq. (9) suggests that the variables $\mathbf{x}_n(t_i)$ and $\mathbf{y}_n(i)$ are independent. However, during the training process, $\mathbf{x}_n(t_i)$ and $\mathbf{y}_n(i)$ are actually dependent. This is because aggregating frozen blocks is not meaningful (if $x_n^k(i) = 0$, then $y_n^k(i) = 0$). Consequently, $\mathbf{y}_n(i)$ depends on $\mathbf{x}_n(t_i)$. Additionally, due to constraint c1, minimizing training latency alone is not enough. We must balance both performance and training time. Since the FL process involves local training first and aggregation later, we adopt a staged optimization: first minimize computation latency under a given communication strategy, and then optimize communication based on the obtained computation decisions. This is an approximate solution to (8), not an exact global optimum.

Based on this analysis, we can try to break down the optimization problem in (9) into two smaller subproblems to find a suboptimal solution. The two subproblems are local training processing (**SubP1**) and global aggregation processing (**SubP2**) for each iteration $i \in \mathcal{I}$.

$$\textbf{SubP1}: \min_{\{\mathbf{x}_n(t_i)\}_{n\in\mathcal{N},t_i\in\mathcal{T}_i}} \left\{ \max_{n\in\mathcal{N}} \left\{ \sum_{t_i\in\mathcal{T}_i} |\mathcal{D}_n|\mu\left(\mathbf{w}_n(t_i)\mathbf{x}_n^\top(t_i)\right) C_n \right\} \right\},$$

$$\text{s.t. c2.} \tag{10}$$

$$\textbf{SubP2}: \min_{\{\mathbf{y}_n(i)\}_{n\in\mathcal{N}}} \left\{ \max_{n\in\mathcal{N}} \left\{ \beta\left(\mathbf{w}_n(i)\mathbf{y}_n^\top(i)\right) R_n \right\} \right\},$$

$$\text{s.t. c3.} \tag{11}$$

The overview of FedMBDP is illustrated in Fig. 3. In the local training processing (**SubP1**), the computation variables, representing which blocks to train, are determined by answering three key questions outlined in Algorithm 2. After completing local model training, the global aggregation processing (**SubP2**) decides which blocks to aggregate as communication variables by comparing them with their previous versions, as described in Algorithm 3. These two subproblems are coupled through the evolving model parameters, and are therefore solved in an alternating manner across communication rounds. Moreover, the global model includes multiple classifiers to evaluate its performance, calculate the patience value, and broadcast this information to all edge servers. Next, we will introduce Algorithms 2 and 3 in detail. In Section V, we provide theoretical guarantees for the convergence under this decomposition, indicating that this approximation does not compromise overall convergence.

B. Block Adaptive Partitioning Algorithm

To ensure generality, block partitioning requires an automated mechanism that analyzes the uninitialized model to identify uncoupled points, thereby eliminating manual design. This mechanism must strictly adhere to two constraints: *i)* The partition strategy must be fixed before training begins. Dynamically shifting boundaries mid-training invalidates existing auxiliary classifiers, necessitating prohibitive retraining from scratch. This dynamic overhead contradicts our efficiency goals, risks aggregation inconsistency, and violates theoretical convergence assumptions (Section V-B). *ii)* All local models must share the same partition strategy. Given their identical architectures, globally uniform partitioning prevents disrupted network connections and subsequent aggregation failures.

To satisfy these constraints, we introduce the Topology-Guided Static Pre-partitioning (TGSP) algorithm (Algorithm 1). Executed entirely at initialization without runtime profiling, TGSP parses the uninitialized model $\mathbf{w}$ into a Directed Acyclic Graph (DAG). It segments the DAG at topological bridges into M atomic modules $\mathcal{A} = [a_1, \ldots, a_M]$, safely preserving internal structures like residual connections. To eliminate micro-blocks—which lack semantic depth and inflate classifier overhead—TGSP calculates the intrinsic

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on October 27,2026 at 23:55:19 UTC from IEEE Xplore. Restrictions apply.

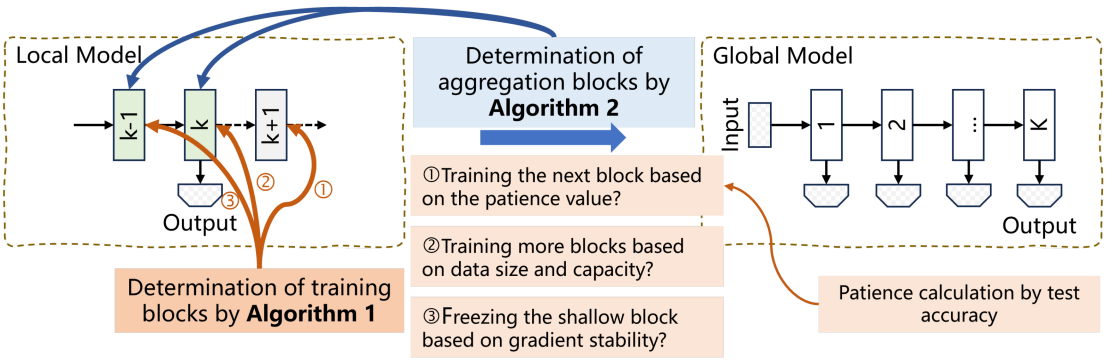


Fig. 3. An overview of FedMBDP.

Algorithm 1: Topology-Guided Static Pre-partitioning.

Input: Uninitialized global model architecture $\mathbf{w}$.
Output: Adaptive partition strategy $\mathcal{K}$.

$\mathcal{G} \leftarrow$ DAG parsed from $\mathbf{w}$;
/* **Safe Segmentation & Weighting** */
Segment $\mathcal{G}$ into M atomic modules $\mathcal{A}$;
Calculate mean weight μ;
/* **Parameter-free Adaptive Grouping** */
Initialize contiguous blocks $\mathcal{B}$;
while $\mathbf{w}(B) < \mu, \exists B \in \mathcal{B}$ *and* $|\mathcal{B}| > 1$ **do**
  $B_{min} \leftarrow \arg\min_{B \in \mathcal{B}} \mathbf{w}(B)$;
  Merge B_{min} with its adjacent block B_{adj};
  Update $\mathcal{B}$ with the newly merged block;
end
return $\mathcal{K}$ based on $\mathcal{B}$;

mean weight $\mu = \frac{1}{M}\sum_{i=1}^{M} \mathbf{w}(a_i)$, where $\mathbf{w}(\cdot)$ counts trainable layers. Initializing $\mathcal{B} = \{\{a_1\}, \ldots, \{a_M\}\}$, it iteratively merges the lightest block with its smaller neighbor until no block weight falls strictly below μ. This parameter-free process naturally yields a balanced partition strategy $\mathcal{K}$ with an automatically determined block count.

C. Local Training Processing Algorithm

The local training processing (i.e., **SubP1**) focuses on determining the computation variable $\mathbf{x}_n(t_i)$, i.e., determining which blocks participate in the training process. Let $\mathcal{K}_n \in \mathcal{K}$ denote the training blocks at edge server n. Two rules govern the local training process:

Remark 1: According to block-wise learning, local models should be trained from shallow to deep blocks. That means local models are trained sequentially from block 1 to K.

Remark 2: To ensure local model participation in training, at least one block $k \in \mathcal{K}$ must be computed for each edge server n at time t_i, i.e., $\sum_{k\in\mathcal{K}} x_n^k(t_i) \geq 1$.

Due to differences in computation capabilities and dataset sizes across edge servers, the computational latency differ among the N edge servers. We need to identify the edge server with the longest computational latency. Considering that if $\mathbf{x}_n(t_i) = \mathbf{x}_m(t_i)$ for $n \neq m$ and $\forall n, m \in \mathcal{N}$, then $\mu(\mathbf{w}_n(t_i)\mathbf{x}_n^\top(t_i)) = \mu(\mathbf{w}_m(t_i)\mathbf{x}_m^\top(t_i))$ also holds true, as these two edge servers share the same model size. Here, $x_n^k(t_i)$ in $\mathbf{x}_n(t_i)$ can be obtained by:

$$x_n^k(t_i) = \begin{cases} 1, \forall k \in \mathcal{K}_n \\ 0, \forall k \in \mathcal{K} \setminus \mathcal{K}_n \end{cases} \quad \forall n \in \mathcal{N}, t_i \in \mathcal{T}_i. \tag{12}$$

Thus, we can identify the edge server n^* with the maximum $|\mathcal{D}_{n^*}|C_{n^*}$ as: [1]

$$n^* = \arg\max_{n\in\mathcal{N}} \{|\mathcal{D}_n|C_n\}. \tag{13}$$

To reduce the waiting latency and enhance learning efficiency, other edge servers $\mathcal{N} \setminus \{n^*\}$ should train additional blocks beyond those trained by n^*. So we consider the computation variable $\mathbf{x}_{n^*}$ as the base decision and $\max \mathcal{K}_m \geq \max \mathcal{K}_{n^*}, m \in \mathcal{N} \setminus \{n^*\}$. Once $\mathbf{x}_{n^*}(t_i)$ is determined, $\mathbf{x}_m(t_i)$ can be determined by the following constraint

$$\sum_{t_i\in\mathcal{T}_i} |\mathcal{D}_{n^*}|\mu\left(\mathbf{w}_{n^*}(t_i)\mathbf{x}_{n^*}^\top(t_i)\right) C_{n^*} \geq$$
$$\sum_{t_i\in\mathcal{T}_i} |\mathcal{D}_m|\mu\left(\mathbf{w}_m(t_i)\mathbf{x}_m^\top(t_i)\right) C_m, \forall m \in \mathcal{N} \setminus \{n^*\}. \tag{14}$$

The decision of when to train block $k+1$ is crucial. Following the approach in [39], we employ a patience mechanism to assess the effectiveness of training block k. A patience value P is assigned, and if the test accuracy of the current global model deteriorates compared to the best test accuracy of the previous global model, P is decremented by one. Once P reaches zero, training proceeds to block $k+1$.

As k increases, the shallower blocks will become more stable [20], with their gradients approaching zero. To reduce FLOPs, we adopt an adaptive freezing mechanism to freeze blocks with smaller gradients [40]. Let $k_{\text{frozen}} = 1$ denote the initial frozen block. Specifically, the frozen block is determined as

$$k_{\text{frozen}} = \min\left\{k_{\text{frozen}} + \gamma(\max \mathcal{K}_{n^*} - k_{\text{frozen}}), \right.$$
$$\left. \arg\min_{k\in\{k_{\text{frozen}},\ldots,\max\mathcal{K}_n\}} \|\mathbf{g}_k(t_i)\| \right\}$$

[1] As each edge server only reports its local data size, there is no risk of data leakage.

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on October 27,2026 at 23:55:19 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2: Local Training Processing.

Input: $|\mathcal{D}_{n^*}|C_{n^*}$ and P.
Output: $\mathbf{x}_n(t_i)$.
/* **Block-Wise Training** */
if $P == 0$ **then**
  Set $k = k + 1$;
  Push block k into $\mathcal{K}_n$;
  Push more blocks into $\mathcal{K}_n$ Eq. (14);
end
/* **Freeze Stable Block** */
if $\max \mathcal{K}_n > K/2$ **then**
  Freeze block k_{frozen} Eq. (15);
  Pop block k_{frozen} from $\mathcal{K}_n$;
end
Calculate and return $\mathbf{x}_n(t_i)$ based on Eq. (12);

$$\text{s.t. } \gamma \in \{0, 1\}, t_i \in \mathcal{T}_i, \forall i \in \mathcal{I}. \tag{15}$$

The pseudocode of the local training processing is shown in Algorithm 2. Steps 1-5 determine whether to train the next block. Steps 6-10 determine whether to freeze the stable block.

D. Global Aggregation Processing Algorithm

The global aggregation processing (i.e., **SubP2**) focuses on the communication variable $\mathbf{y}_n(i)$. During each training round i, each edge server maintains the latest global model $\mathbf{w}(i-1)$. After local training at round i, edge server n compares $\mathbf{w}_n^k(T_i)$ of block $k \in \mathcal{K}_n$ with $\mathbf{w}^k(i-1)$ based on the Mean Block Difference (MBD) metric:

$$S(\mathbf{w}^k(i-1), \mathbf{w}_n^k(T_i)) = \frac{\left\|\mathbf{w}^k(i-1) - \mathbf{w}_n^k(T_i)\right\|_2}{|\mathbf{w}_n^k(T_i)|}, \quad \forall i \in \mathcal{I} \setminus \{1\}, k \in \mathcal{K}_n, n \in \mathcal{N}. \tag{16}$$

Only uploading the training blocks $\mathcal{K}_n$ is meaningful, obtained by Algorithm 2. Let $\mathcal{K}_n^{\text{Upload}} \leftarrow \mathcal{K}_n$ denote the set of blocks to be uploaded. If the MBD of each block from $\mathcal{K}_n$ is small, i.e., less than the minimal difference threshold S^*, the block will not be uploaded. The pseudocode of the global aggregation processing is shown in Algorithm 3. Step 1 accesses the current computation variables. Steps 2-7 determine which local blocks to be uploaded.

The pseudocode of FedMBDP is presented in Algorithm 4, which consists of three key components: *i)* block partitioning in Algorithm 1, *ii)* local training in Algorithm 2, and *iii)* global aggregation in Algorithm 3. Specifically, Step 1 adaptively partitions the model into K blocks, while Step 3 identifies the edge server with the slowest training progress. In Steps 4–5, each local model is trained following Algorithm 2 to minimize computational costs while maintaining performance. Upon completing each local training iteration, Steps 5–6 determine which blocks to aggregate based on Algorithm 3.

Algorithm 3: Global Aggregation Processing.

Input: $\mathcal{K}_n$ and $\mathbf{w}(i-1)$.
Output: $\mathbf{y}_n(i)$.
1 $\mathcal{K}_n^{\text{Upload}} \leftarrow \mathcal{K}_n$;
/* **Do Not Upload Similar Blocks** */
2 **for** $k \in \mathcal{K}_n$ **do**
3 Calculate $S(\mathbf{w}^k(i-1), \mathbf{w}_n^k(T_i))$ Eq. (16);
4 **if** $S(\mathbf{w}^k(i-1), \mathbf{w}_n^k(T_i)) < S^*$ **then**
5 Pop k from $\mathcal{K}_n^{\text{Upload}}$;
6 **end**
7 **end**
8 Calculate and return $\mathbf{y}_n(i)$ based on Eq. (12);

Algorithm 4: FedMBDP.

1 Determine $\mathcal{K}$ by **Algorithm 1**;
2 Initialize local models and the global model;
3 Calculate $|\mathcal{D}_{n^*}|C_{n^*}$ according to Eq. (13);
4 **while** $i \in \mathcal{I}$ **do**
  /* **Local Training** */
  for $n \in \mathcal{N}$ *in parallel* **do**
    $\mathbf{w}_n(i)\mathbf{y}_n^\top(i) \leftarrow \mathbf{w}(i)\mathbf{y}_n^\top(i)$ where $\mathbf{w}(i)$ is downloaded from the cloud server;
    for $t_i \in \mathcal{T}_i$ **do**
      Determine $\mathbf{x}_n(t_i)$ by **Algorithm 2**;
      Train local model $\mathbf{w}_n(t_i)\mathbf{x}_n^\top(t_i)$ by (3);
      Get $l_n^{\text{cpt}}(t_i)$ according to Eq. (5);
    end
  end
5
  /* **Global Aggregate** */
  for $n \in \mathcal{N}$ *in parallel* **do**
    Determine $\mathbf{y}_n(i)$ by **Algorithm 3**;
    Upload $\mathbf{w}_n(i)\mathbf{y}_n^\top(i)$ and the local computation latency $\sum_{t \in \mathcal{T}_i} l_n^{\text{cpt}}(t_i)$;
    Get $l_n^{\text{cmt}}(i)$ Eq. (6);
  end
  Aggregate $\mathbf{w}(i)$ according to Eq. (4);
  Test the global model and calculate the patience number P;
6
7 **end**

E. Complexity Analysis

To analyze the computational complexity of FedMBDP, we assess Algorithms 2, 3, and 4. For Algorithm 2, since there is no loop, its complexity is constant at $\mathcal{O}(1)$. For Algorithm 3, the complexity is proportional to the number of elements in $\mathcal{K}_n$, leading to $\mathcal{O}(|\mathcal{K}_n|)$. For Algorithm 4, which includes model training, the number of computations varies depending on the number of active training blocks. Denoting Φ as the number of multiplications required to train the whole local model on one sample, and B as the batch size, the maximum computational complexity for training the entire model is $\mathcal{O}(B\Phi)$. Thus, the complexity for local training across all

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on October 27,2026 at 23:55:19 UTC from IEEE Xplore. Restrictions apply.

devices $|\mathcal{N}|$ and time steps $|\mathcal{T}_i|$ is $\mathcal{O}(|\mathcal{N}||\mathcal{T}_i|B\Phi)$. The complexity of the global aggregation step is $\mathcal{O}(|\mathcal{N}||\mathcal{K}_n|)$, based on the number of devices and parameters. It is easy to know $\mathcal{O}(|\mathcal{N}||\mathcal{K}_n|) < \mathcal{O}(|\mathcal{N}||\mathcal{T}_i|B\Phi)$. Therefore, the maximum computational complexity of `FedMBDP` across all iterations $|\mathcal{I}|$ is $\mathcal{O}(|\mathcal{I}||\mathcal{N}||\mathcal{T}_i|B\Phi)$. In practice, since we only train parts of a local model via block-wise learning and freezing, the computational complexity of `FedMBDP` is less than $\mathcal{O}(|\mathcal{I}||\mathcal{N}||\mathcal{T}_i|B\Phi)$.

V. Convergence Analysis

A. Problem Setup and Round-Level Abstraction

Recall that the global model on the server is $\mathbf{w}(i) = (\mathbf{w}^1(i), \ldots, \mathbf{w}^K(i))$, where $\mathbf{w}^k(i)$ denotes the parameters of block k at round i. Each client $n \in \mathcal{N}$ maintains a local model $\mathbf{w}_n(i) = (\mathbf{w}_n^1(i), \ldots, \mathbf{w}_n^K(i))$. The local update and global aggregation follow (3) and (4), respectively.

We analyze a fixed block k. Due to the progressive block-wise learning and freezing in Algorithm 2, the input to block k is the output of block $k-1$, whose distribution can drift across rounds. Let $z \in \mathcal{Z}$ denote the input features to block k. We denote by $p_{k-1}^i(z)$ the input distribution at round i and by $p_{k-1}^*(z)$ the stationary distribution once block $k-1$ stabilizes.

Define the (stationary) block-k objective,

$$F_k(\mathbf{w}^k) := \mathbb{E}_{z\sim p_{k-1}^*}\left[\ell(\mathbf{w}^k; z)\right]. \tag{17}$$

For convergence we adopt a standard round-level abstraction in FL: the effect of the T_i local steps within round i for block k is represented by an effective global stepsize $\eta_i > 0$. Thus the block-k global update of `FedMBDP` is written as

$$\mathbf{w}^k(i+1) = \mathbf{w}^k(i) - \eta_i \mathbf{g}_i, \tag{18}$$

where $\mathbf{g}_i$ is the block-wise gradient estimator aggregated from uploading clients.

Remark 3: Round-level abstraction: Following the standard communication-round analysis of FL, we equivalently represent the effect of the T_i local updates of block k within round i by the effective stepsize η_i and the round-level block gradient estimator $\mathbf{g}_i$ in (18).

B. Assumptions

Assumption 1 (Smoothness): F_k is differentiable and L-smooth. Equivalently, ∇F_k is L-Lipschitz:

$$\|\nabla F_k(\mathbf{u}) - \nabla F_k(\mathbf{v})\|_2 \le L\|\mathbf{u}-\mathbf{v}\|_2, \ \forall \mathbf{u}, \mathbf{v}. \tag{19}$$

Assumption 2 (Bounded second moment): There exists $G > 0$ such that for any round i,

$$\mathbb{E}_{z\sim p_{k-1}^i}\left[\|\nabla \ell(\mathbf{w}^k; z)\|_2^2\right] \le G^2. \tag{20}$$

Assumption 3 (Input distribution drift): Define the input distribution drift as

$$c_{k-1}^i := \int_{\mathcal{Z}} \left|p_{k-1}^i(z) - p_{k-1}^*(z)\right| \mathrm{d}z. \tag{21}$$

We assume

$$\sum_{i=1}^{I} \sqrt{c_{k-1}^i} = \mathcal{O}\left(\sqrt{I}\right). \tag{22}$$

Remark 4: Interpretation of Assumption 3: Assumption 3 states that the cumulative input distribution drift is sublinear in the number of rounds. The notation $\mathcal{O}(\sqrt{I})$ means that there exists a constant $C > 0$ such that $\sum_{i=1}^{I} \sqrt{c_{k-1}^i} \le C\sqrt{I}$ for sufficiently large I. This is mild under the patience-based progressive training, as the preceding block tends to stabilize over rounds; e.g., if $c_{k-1}^i = \mathcal{O}(1/i)$, then $\sum_{i=1}^{I} \sqrt{c_{k-1}^i} = \mathcal{O}(\sqrt{I})$.

Assumption 4 (MBD-induced gradient selection bias): Let $\mathcal{N}_i^k$ be the set of clients uploading block k at round i according to MBD (16) with threshold S^*. There exists $\kappa_g > 0$ such that

$$\Big\| \sum_{n\in\mathcal{N}} p_n \nabla F_{n,k}(\mathbf{w}_n^k(i)) - \sum_{n\in\mathcal{N}_i^k} p_n \nabla F_{n,k}(\mathbf{w}_n^k(i)) \Big\|_2 \le \kappa_g S^*. \tag{23}$$

Assumption 4 is aligned with the design of MBD, where clients skip uploading block k only when its evolution is sufficiently small, so the aggregated block-k gradient computed from uploading clients is a close approximation of the full aggregated gradient.

Remark 5: Interpretation of Assumption 4: Since MBD filters out only those block updates with sufficiently small evolution, the induced bias in the aggregated block gradient is bounded by a threshold-controlled term. This assumption formalizes the algorithmic design intent and is validated empirically by the ablation results in Section VI-E.

C. A Drift-Induced Gradient Deviation Lemma

Lemma 1 (Gradient deviation under input drift): Under Assumptions 2 and 3, for any round i,

$$\left\|\mathbb{E}_{z\sim p_{k-1}^i}\left[\nabla \ell(\mathbf{w}^k(i); z)\right] - \nabla F_k(\mathbf{w}^k(i))\right\|_2 \le G\sqrt{2c_{k-1}^i}. \tag{24}$$

Proof: Let $\nabla \ell(z) := \nabla \ell(\mathbf{w}^k(i); z)$. Then

$$\begin{aligned} &\left\|\mathbb{E}_{p_{k-1}^i}[\nabla \ell(z)] - \mathbb{E}_{p_{k-1}^*}[\nabla \ell(z)]\right\|_2 \\ &= \left\| \int_{\mathcal{Z}} \nabla \ell(z) \left(p_{k-1}^i(z) - p_{k-1}^*(z)\right) \mathrm{d}z \right\|_2 \\ &\le \int_{\mathcal{Z}} \|\nabla \ell(z)\|_2 \left|p_{k-1}^i(z) - p_{k-1}^*(z)\right| \mathrm{d}z. \end{aligned} \tag{25}$$

Let $q(z) := |p_{k-1}^i(z) - p_{k-1}^*(z)| \ge 0$. Applying the Cauchy–Schwarz inequality under the measure $q(z)\mathrm{d}z$, we obtain

$$\begin{aligned} &\int_{\mathcal{Z}} \|\nabla \ell(z)\|_2 \, q(z) \, \mathrm{d}z \\ &\le \sqrt{\int_{\mathcal{Z}} \|\nabla \ell(z)\|_2^2 \, q(z) \, \mathrm{d}z} \sqrt{\int_{\mathcal{Z}} q(z) \, \mathrm{d}z}. \end{aligned} \tag{26}$$

By definition,

$$\int_{\mathcal{Z}} q(z) \, \mathrm{d}z = \int_{\mathcal{Z}} \left|p_{k-1}^i(z) - p_{k-1}^*(z)\right| \mathrm{d}z = c_{k-1}^i. \tag{27}$$

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on October 27,2026 at 23:55:19 UTC from IEEE Xplore. Restrictions apply.

Moreover, Assumption 2 implies that there exists $G > 0$ such that for any round i,

$$\mathbb{E}_{z\sim p^i_{k-1}}\left[\|\nabla \ell(\mathbf{w}^k; z)\|_2^2\right] \le G^2,$$
$$\mathbb{E}_{z\sim p^*_{k-1}}\left[\|\nabla \ell(\mathbf{w}^k; z)\|_2^2\right] \le G^2. \tag{28}$$

Using the inequality $|p^i_{k-1}(z) - p^*_{k-1}(z)| \le p^i_{k-1}(z) + p^*_{k-1}(z)$, we have

$$\begin{aligned}&\int_{\mathcal{Z}} \|\nabla \ell(z)\|_2^2 \, |p^i_{k-1}(z) - p^*_{k-1}(z)| \, \mathrm{d}z \\ &\le \int_{\mathcal{Z}} \|\nabla \ell(z)\|_2^2 \, p^i_{k-1}(z) \, \mathrm{d}z + \int_{\mathcal{Z}} \|\nabla \ell(z)\|_2^2 \, p^*_{k-1}(z) \, \mathrm{d}z \\ &\le 2G^2. \end{aligned} \tag{29}$$

Combining the above bounds with (26) yields

$$\left\|\mathbb{E}_{z\sim p^i_{k-1}}[\nabla \ell(\mathbf{w}^k(i); z)] - \nabla F_k(\mathbf{w}^k(i))\right\|_2 \le G\sqrt{2c^i_{k-1}}, \tag{30}$$

which completes the proof of (24). □

D. Gradient Estimator Decomposition

Let $\mathbf{g}^k_{n,i}$ be the stochastic gradient of block k on client n computed at the end of the local procedure of round i. The MBD-based global estimator is

$$\mathbf{g}_i := \sum_{n\in\mathcal{N}^k_i} p_n \mathbf{g}^k_{n,i}. \tag{31}$$

Define the full aggregated stochastic gradient

$$\tilde{\mathbf{g}}_i := \sum_{n\in\mathcal{N}} p_n \mathbf{g}^k_{n,i}. \tag{32}$$

We further define the conditional expectations:

$$\mathbf{G}_i := \mathbb{E}\left[\tilde{\mathbf{g}}_i \mid \mathbf{w}^k(i)\right], \qquad \mathbf{H}_i := \mathbb{E}\left[\mathbf{g}_i \mid \mathbf{w}^k(i)\right]. \tag{33}$$

Then the difference between $\mathbf{H}_i$ and the true gradient $\nabla F_k(\mathbf{w}^k(i))$ can be decomposed into:

$$\mathbf{H}_i = \nabla F_k(\mathbf{w}^k(i)) + \mathbf{e}^{\text{drift}}_i + \mathbf{e}^{\text{comm}}_i, \tag{34}$$

where

$$\mathbf{e}^{\text{drift}}_i := \mathbf{G}_i - \nabla F_k(\mathbf{w}^k(i)), \tag{35}$$
$$\mathbf{e}^{\text{comm}}_i := \mathbf{H}_i - \mathbf{G}_i. \tag{36}$$

By Lemma 1,

$$\|\mathbf{e}^{\text{drift}}_i\|_2 \le G\sqrt{2c^i_{k-1}}. \tag{37}$$

By Assumption 4,

$$\|\mathbf{e}^{\text{comm}}_i\|_2 \le \kappa_g S^*. \tag{38}$$

Finally, define the zero-mean stochastic noise:

$$\boldsymbol{\xi}_i := \mathbf{g}_i - \mathbf{H}_i, \qquad \mathbb{E}\left[\boldsymbol{\xi}_i \mid \mathbf{w}^k(i)\right] = \mathbf{0}. \tag{39}$$

Thus,

$$\mathbf{g}_i = \nabla F_k(\mathbf{w}^k(i)) + \mathbf{e}^{\text{drift}}_i + \mathbf{e}^{\text{comm}}_i + \boldsymbol{\xi}_i. \tag{40}$$

E. Main Convergence Theorem

Theorem 1 (Convergence of: `FedMBDP` **for block** k*)* Suppose Assumptions 1–4 hold. Choose a decaying stepsize $\eta_i = \sqrt{\eta_0/(i+1)}$ with $\eta_0 > 0$ and an MBD threshold $S^* = \mathcal{O}(1/\sqrt{I})$. If the drift sequence satisfies (22), then after I rounds,

$$\min_{i\in\{1,\dots,I\}} \mathbb{E}\left[\|\nabla F_k(\mathbf{w}^k(i))\|_2^2\right] \le \mathcal{O}\left(\frac{1}{\sqrt{I}}\right) + \mathcal{O}(S^*) + \mathcal{O}\left(\frac{1}{I}\sum_{i=1}^{I}\sqrt{c^i_{k-1}}\right). \tag{41}$$

Proof: Step 1: One-step descent via smoothness: By the L-smoothness of F_k (Assumption 1), for any $\mathbf{u}$ and $\mathbf{v}$,

$$F_k(\mathbf{u}) \le F_k(\mathbf{v}) + \left\langle \nabla F_k(\mathbf{v}), \mathbf{u} - \mathbf{v}\right\rangle + \frac{L}{2}\|\mathbf{u} - \mathbf{v}\|_2^2. \tag{42}$$

Setting $\mathbf{u} = \mathbf{w}^k(i+1)$ and $\mathbf{v} = \mathbf{w}^k(i)$ and subtracting $F_k(\mathbf{w}^k(i))$ from both sides yields

$$\begin{aligned}F_k(\mathbf{w}^k(i+1)) - F_k(\mathbf{w}^k(i)) &\le \frac{L}{2}\|\mathbf{w}^k(i+1) - \mathbf{w}^k(i)\|_2^2 \\ &+ \left\langle \nabla F_k(\mathbf{w}^k(i)), \mathbf{w}^k(i+1) - \mathbf{w}^k(i)\right\rangle. \end{aligned} \tag{43}$$

Substituting the round-level update (18) gives

$$\begin{aligned}F_k(\mathbf{w}^k(i+1)) - F_k(\mathbf{w}^k(i)) \le &-\eta_i \left\langle \nabla F_k(\mathbf{w}^k(i)), \mathbf{g}_i\right\rangle \\ &+ \frac{L\eta_i^2}{2}\|\mathbf{g}_i\|_2^2. \end{aligned} \tag{44}$$

Step 2: Take conditional expectation: Taking $\mathbb{E}[\cdot \mid \mathbf{w}^k(i)]$ on both sides of (44), we have

$$\begin{aligned}&\mathbb{E}\left[F_k(\mathbf{w}^k(i+1)) \mid \mathbf{w}^k(i)\right] - F_k(\mathbf{w}^k(i)) \le \\ &-\eta_i \left\langle \nabla F_k(\mathbf{w}^k(i)), \mathbf{H}_i\right\rangle + \frac{L\eta_i^2}{2}\mathbb{E}\left[\|\mathbf{g}_i\|_2^2 \mid \mathbf{w}^k(i)\right]. \end{aligned} \tag{45}$$

Using (34),

$$\begin{aligned}\left\langle \nabla F_k(\mathbf{w}^k(i)), \mathbf{H}_i\right\rangle &= \|\nabla F_k(\mathbf{w}^k(i))\|_2^2 \\ &+ \left\langle \nabla F_k(\mathbf{w}^k(i)), \mathbf{e}^{\text{drift}}_i + \mathbf{e}^{\text{comm}}_i\right\rangle. \end{aligned} \tag{46}$$

Plugging (46) into (45) yields

$$\begin{aligned}\mathbb{E}\left[F_k(\mathbf{w}^k(i+1)) \mid \mathbf{w}^k(i)\right] \le &F_k(\mathbf{w}^k(i)) - \eta_i\|\nabla F_k(\mathbf{w}^k(i))\|_2^2 \\ &- \eta_i \left\langle \nabla F_k(\mathbf{w}^k(i)), \mathbf{e}^{\text{drift}}_i + \mathbf{e}^{\text{comm}}_i\right\rangle \\ &+ \frac{L\eta_i^2}{2}\mathbb{E}\left[\|\mathbf{g}_i\|_2^2 \mid \mathbf{w}^k(i)\right]. \end{aligned} \tag{47}$$

Step 3: Bound the cross terms by Young's inequality: For any vectors a, b and any $\varepsilon > 0$, $\langle a, b\rangle \le \frac{\varepsilon}{2}\|a\|^2 + \frac{1}{2\varepsilon}\|b\|^2$. Applying with $a = \nabla F_k(\mathbf{w}^k(i))$, we get

$$\begin{aligned}&-\eta_i \langle \nabla F_k(\mathbf{w}^k(i)), \mathbf{e}^{\text{drift}}_i\rangle \\ &\le \eta_i\left(\frac{\varepsilon}{2}\|\nabla F_k(\mathbf{w}^k(i))\|_2^2 + \frac{1}{2\varepsilon}\|\mathbf{e}^{\text{drift}}_i\|_2^2\right), \end{aligned} \tag{48}$$

$$-\eta_i \langle \nabla F_k(\mathbf{w}^k(i)), \mathbf{e}^{\text{comm}}_i\rangle$$

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on October 27,2026 at 23:55:19 UTC from IEEE Xplore. Restrictions apply.

$$\leq \eta_i \left(\frac{\varepsilon}{2} \|\nabla F_k(\mathbf{w}^k(i))\|_2^2 + \frac{1}{2\varepsilon} \|\mathbf{e}_i^{\text{comm}}\|_2^2 \right). \tag{49}$$

Using (37)–(38), we have

$$\|\mathbf{e}_i^{\text{drift}}\|_2^2 \leq 2G^2 c_{k-1}^i, \tag{50}$$

$$\|\mathbf{e}_i^{\text{comm}}\|_2^2 \leq \kappa_g^2 (S^*)^2. \tag{51}$$

Step 4: Bound the second moment of $\mathbf{g}_i$*:* From (40),

$$\begin{aligned} \|\mathbf{g}_i\|_2^2 &\leq 2\|\nabla F_k(\mathbf{w}^k(i))\|_2^2 + 2\|\mathbf{e}_i^{\text{drift}} + \mathbf{e}_i^{\text{comm}} + \boldsymbol{\xi}_i\|_2^2 \\ &\leq 2\|\nabla F_k(\mathbf{w}^k(i))\|_2^2 + 4\|\mathbf{e}_i^{\text{drift}}\|_2^2 + 4\|\mathbf{e}_i^{\text{comm}}\|_2^2 \\ &\quad + 4\|\boldsymbol{\xi}_i\|_2^2. \end{aligned} \tag{52}$$

Taking conditional expectation and using Assumption 2, we obtain

$$\mathbb{E}\left[\|\boldsymbol{\xi}_i\|_2^2 \mid \mathbf{w}^k(i)\right] \leq G^2. \tag{53}$$

Thus,

$$\begin{aligned} \mathbb{E}\left[\|\mathbf{g}_i\|_2^2 \mid \mathbf{w}^k(i)\right] &\leq 2\|\nabla F_k(\mathbf{w}^k(i))\|_2^2 + 8G^2 c_{k-1}^i \\ &\quad + 4\kappa_g^2 (S^*)^2 + 4G^2. \end{aligned} \tag{54}$$

Step 5: Combine bounds: Substituting (48)–(49) and (54) into (47), we collect the coefficients of $\|\nabla F_k(\mathbf{w}^k(i))\|_2^2$:

$$-\eta_i \left(1 - \varepsilon - 2L\eta_i\right) \|\nabla F_k(\mathbf{w}^k(i))\|_2^2. \tag{55}$$

Choosing $\varepsilon = 1/4$ and assuming the stepsize satisfies $\eta_i \leq \frac{1}{8L}$ (so that $2L\eta_i \leq 1/4$), the coefficient becomes at most $-\frac{\eta_i}{2}$. Thus, we have

$$\begin{aligned} \mathbb{E}\left[F_k(\mathbf{w}^k(i+1)) \mid \mathbf{w}^k(i)\right] &\leq F_k(\mathbf{w}^k(i)) - \frac{\eta_i}{2} \|\nabla F_k(\mathbf{w}^k(i))\|_2^2 \\ &\quad + C_a \eta_i c_{k-1}^i + C_b \eta_i (S^*)^2 + C_c \eta_i^2. \end{aligned} \tag{56}$$

Step 6: Sum over rounds: Taking the total expectation on both sides of (56) (noting that $\mathbb{E}[\mathbb{E}[\cdot|\mathbf{w}^k(i)]] = \mathbb{E}[\cdot]$) and rearranging the terms to isolate $\|\nabla F_k(\mathbf{w}^k(i))\|_2^2$ yields

$$\begin{aligned} \frac{\eta_i}{2} \mathbb{E}\left[\|\nabla F_k(\mathbf{w}^k(i))\|_2^2\right] &\leq \mathbb{E}\left[F_k(\mathbf{w}^k(i))\right] - \mathbb{E}\left[F_k(\mathbf{w}^k(i+1))\right] \\ &\quad + C_a \eta_i c_{k-1}^i + C_b \eta_i (S^*)^2 + C_c \eta_i^2. \end{aligned} \tag{57}$$

Multiplying both sides by 2 and summing over $i = 1, \ldots, I$, we utilize the telescoping sum property $\sum_{i=1}^I (\mathbb{E}[F_k(\mathbf{w}^k(i))] - \mathbb{E}[F_k(\mathbf{w}^k(i+1))]) = F_k(\mathbf{w}^k(1)) - \mathbb{E}[F_k(\mathbf{w}^k(I+1))]$. Since $F_k(\mathbf{w}^k(I+1)) \geq F^*$ (where F^* is the global minimum), we obtain

$$\begin{aligned} \sum_{i=1}^I \eta_i \mathbb{E}\left[\|\nabla F_k(\mathbf{w}^k(i))\|_2^2\right] &\leq 2\left(F_k\left(\mathbf{w}^k(1)\right) - F^*\right) \\ &\quad + C_1 \sum_{i=1}^I \eta_i c_{k-1}^i + C_2 (S^*)^2 \sum_{i=1}^I \eta_i + C_3 \sum_{i=1}^I \eta_i^2, \end{aligned} \tag{58}$$

where we define $C_1 = 2C_a$, $C_2 = 2C_b$, and $C_3 = 2C_c$. Using the inequality $(\min_j x_j)(\sum \eta_j) \leq \sum \eta_j x_j$, we have

$$\min_{1\leq i\leq I} \mathbb{E}\left[\|\nabla F_k(\mathbf{w}^k(i))\|_2^2\right] \sum_{i=1}^I \eta_i \leq \sum_{i=1}^I \eta_i \mathbb{E}\left[\|\nabla F_k(\mathbf{w}^k(i))\|_2^2\right]. \tag{59}$$

Finally, dividing (58) by $\sum_{i=1}^I \eta_i$ gives

$$\begin{aligned} \min_{1\leq i\leq I} \mathbb{E}\left[\|\nabla F_k(\mathbf{w}^k(i))\|_2^2\right] &\leq \frac{2\left(F_k(\mathbf{w}^k(1)) - F^*\right)}{\sum_{i=1}^I \eta_i} \\ &\quad + C_1 \frac{\sum_{i=1}^I \eta_i c_{k-1}^i}{\sum_{i=1}^I \eta_i} + C_2 (S^*)^2 \\ &\quad + C_3 \frac{\sum_{i=1}^I \eta_i^2}{\sum_{i=1}^I \eta_i}. \end{aligned} \tag{60}$$

Step 7: Plug in the decaying stepsize: With $\eta_i = \sqrt{\eta_0/(i+1)}$,

$$\sum_{i=1}^I \eta_i = \Theta\left(\sqrt{I}\right), \qquad \sum_{i=1}^I \eta_i^2 = \Theta(\log I). \tag{61}$$

Moreover,

$$\begin{aligned} \frac{\sum_{i=1}^I \eta_i c_{k-1}^i}{\sum_{i=1}^I \eta_i} &\leq \frac{\sum_{i=1}^I \eta_i \sqrt{c_{k-1}^i}}{\sum_{i=1}^I \eta_i} \cdot \max_{1\leq i\leq I} \sqrt{c_{k-1}^i} \\ &\leq \mathcal{O}\left(\frac{1}{I} \sum_{i=1}^I \sqrt{c_{k-1}^i}\right), \end{aligned} \tag{62}$$

and Assumption 3 guarantees that the drift contribution is of order $\mathcal{O}(\frac{1}{I}\sum_{i=1}^I \sqrt{c_{k-1}^i})$ after normalization.

Finally, choosing $S^* = \mathcal{O}(1/\sqrt{I})$ and absorbing constants yield

$$\begin{aligned} \min_{1\leq i\leq I} \mathbb{E}\left[\|\nabla F_k(\mathbf{w}^k(i))\|_2^2\right] &\leq \mathcal{O}\left(\frac{1}{\sqrt{I}}\right) + \mathcal{O}(S^*) \\ &\quad + \mathcal{O}\left(\frac{1}{I} \sum_{i=1}^I \sqrt{c_{k-1}^i}\right), \end{aligned} \tag{63}$$

which proves (41). □

VI. Experimental Evaluation

A. Experiment Settings

We evaluate the proposed `FedMBDP` using PyTorch 1.11 on a server with 4 NVIDIA A100 GPUs, 1 AMD EPYC CPU, and 252 GB of memory. The experiments are conducted for two image classification tasks on CIFAR10 and CIFAR100 datasets[2] using three classical models ResNet18, ResNet20, and DenseNet40. We conduct experiments on IID and Non-IID distributions based on the Dirichlet function.

[2]CIFAR10 consists of 50,000 training images and 10,000 testing images in 10 classes. CIFAR-100 consists of 50,000 training images and 10,000 testing images in 100 classes. https://www.cs.toronto.edu/~kriz/cifar.html

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on October 27,2026 at 23:55:19 UTC from IEEE Xplore. Restrictions apply.

During the experiments, we consider a scenario with 100 edge servers and one cloud server, where 10 edge servers actively participate in each global round. The computation frequencies and transmission rates of edge servers are in the ranges [1.8, 2.2] GHz and 10 MBps, respectively. We use the stochastic gradient descent (SGD) optimizer with a learning rate of 0.05 and momentum of 0.9. The training batch size is set to 128. For CIFAR-10, we run 100 global rounds, and for CIFAR-100, we run 200 global rounds, with each round consisting of 5 epochs. Standard image augmentation techniques, such as random cropping and horizontal and vertical flips, are applied to all datasets. The test accuracy is evaluated using the global model on the global test set. The computation cost includes FLOPs for both forward and backward propagation. The FLOPs for forward propagation are measured using the torchprofile function[3] while the FLOPs for backward propagation are measured using the FlopCounterMode function in 'torch.utils.flop_counter'. Please refer to [25] for other detailed parameter settings. Moreover, we compared FedMBDP with six state-of-the-art baselines:

1) *FedAvg [21]* aggregates model updates from distributed clients through weighted averaging.
2) *HeteroFL [24]* accommodates diverse client capabilities by enabling the training of different-sized models across clients with varying computational resources, promoting flexibility in FL.
3) *SLT [25]* introduces a progressive parameter freezing strategy, enabling devices to successively freeze and train portions of the FL model, thus reducing local resource consumption during training.
4) *FedRolex [41]* optimizes learning efficiency by dynamically adjusting client roles based on their resource availability, ensuring a balanced and adaptive FL process.
5) *FD [42]* enhances client participation by leveraging dropout techniques, allowing clients with limited resources to train partial models locally.
6) *Fjord [30]* selectively transmits the most informative model updates, thereby reducing the frequency and size of data exchanged during training.

B. Partition of the Selected Models

To ensure the validity of independent block-wise training, we apply the proposed TGSP algorithm to automatically partition the selected models. Instead of relying on manual heuristics, TGSP parses the computational graph to identify topological bridges. This mathematically guarantees that partitions are exclusively placed at natural modular boundaries (uncoupled points), thereby preserving functional integrity and effective gradient flow. For instance, TGSP naturally avoids arbitrarily bisecting interdependent layers, such as the residual (skip) connections in ResNet architectures or the tightly coupled paths within DenseNet blocks.

By iteratively grouping these atomic modules based on their intrinsic mean weight, TGSP automatically determines the optimal workload-balanced block count for each specific architecture. Consequently, ResNet18, ResNet20, and DenseNet40 are naturally partitioned into $K = 4$, 3, and 3 blocks, respectively (illustrated in Fig. 4). As these automatically generated partitions strictly align with inherent structural motifs, each block remains a functionally consistent sub-unit capable of learning stable feature representations independently. The effectiveness and stability of this automated partition strategy are corroborated by the convergence results shown in Fig. 7.

[3]https://github.com/zhijian-liu/torchprofile

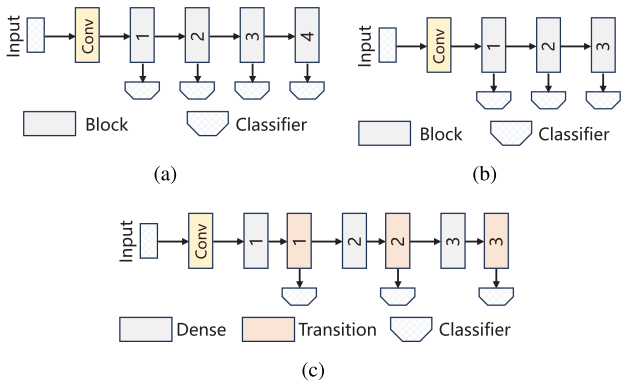


Fig. 4. Partition of Three Models: (a) ResNet18; (b) ResNet20; (c) DenseNet40.

C. Performance Under Different Models

Table II shows the performance of six algorithms on ResNet18. The results show that FedMBDP achieves a competitive accuracy while significantly reducing training latency. On the CIFAR-10 dataset, FedMBDP achieves a high test accuracy of 0.78, which is on par with FedAvg and HeteroFL, while significantly outperforming FD (4.3%↑), FedRolex (20.15%↑), and Fjord (5.41%↑). FedMBDP reduces the total training latency by 26.37% compared to the best baseline FD (26011.33 s vs 19151.89 s), consistent with the abstract. For the CIFAR-100 dataset, FedMBDP has the highest test accuracy of 0.45 among all compared algorithms, indicating its robustness and effectiveness in handling challenging classification tasks. This superior performance is attributed to FedMBDP's ability to enable the shallower blocks of ResNet18 to effectively learn feature representations from the datasets. In terms of training latency, FedMBDP shows a substantial advantage. It records a training latency of 19151.89 seconds for CIFAR-10, considerably lower than the other methods, and 41314.51 seconds for CIFAR-100, again significantly less than its competitors. This reduction in training latency, coupled with high accuracy, underscores the efficiency and practicality of FedMBDP in FL environments.

Table III presents the performance of seven algorithms on ResNet20. The experimental results demonstrate that our proposed algorithm, FedMBDP, achieves notable performance in terms of both test accuracy and training latency. Although FedMBDP does not achieve the highest test accuracy among the comparison methods, it stands out for its remarkably low training latency. On the CIFAR-10 dataset, FedMBDP records a training latency of 5644.59 seconds, significantly lower than FedAvg (37474.01 seconds) and SLT (15428.69 seconds). While

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on October 27,2026 at 23:55:19 UTC from IEEE Xplore. Restrictions apply.

TABLE II
PERFORMANCE OF SIX ALGORITHMS ON RESNET18. THE PERCENTAGES IN THE TABLE INDICATE THE DEGREE OF INCREASE AND DECREASE OF FedMBDP COMPARED TO THE BASELINE. GFLOPS $= 10^9 \times$FLOPS

	Algorithm	Test Accuracy		Training Latency (s)		Computation (GFLOPs)		Communication (MB)	
CIFAR10	FedMBDP	**0.78**		**19151.89**		**37849.04**		**2273.74**	
	FedAvg	**0.78**	0.00	32220.71	-40.56 %↓	63587.58	-40.48 %↓	4269.13	-46.74 %↓
	FD	0.75	4.30 %↑	26011.33	-26.37 %↓	51333.43	-26.27 %↓	3446.14	-34.02 %↓
	FedRolex	0.65	20.15 %↑	26011.33	-26.37 %↓	51333.43	-26.27 %↓	3446.14	-34.02 %↓
	Fjord	0.74	5.41 %↑	26011.33	-26.37 %↓	51333.43	-26.27 %↓	3446.14	-34.02 %↓
	HeteroFL	**0.78**	0.00	26011.33	-26.37 %↓	51333.43	-26.27 %↓	3446.14	-34.02 %↓
CIFAR100	FedMBDP	**0.45**		**41314.51**		**81560.56**		**5342.30**	
	FedAvg	0.43	3.30 %↑	64525.57	-35.97 %↓	127336.45	-35.95 %↓	8573.48	-37.69 %↓
	FD	0.39	14.71 %↑	52098.28	-20.70 %↓	102811.77	-20.67 %↓	6923.93	-22.84 %↓
	FedRolex	0.35	26.89 %↑	52098.28	-20.70 %↓	102811.77	-20.67 %↓	6923.93	-22.84 %↓
	Fjord	0.40	11.50 %↑	52098.28	-20.70 %↓	102811.77	-20.67 %↓	6923.93	-22.84 %↓
	HeteroFL	0.44	2.11 %↑	52098.28	-20.70 %↓	102811.77	-20.67 %↓	6923.93	-22.84 %↓

TABLE III
PERFORMANCE OF SEVEN ALGORITHMS ON RESNET20

	Algorithm	Test Accuracy		Training Latency (s)		Computation (GFLOPs)		Communication (MB)	
CIFAR10	FedMBDP	0.72		**5644.59**		**11285.97**		**16.00**	
	FedAvg	0.79	-8.55 %↓	37474.01	-84.94 %↓	74925.5	-84.94 %↓	112.48	-85.78 %↓
	FD	0.64	12.32 %↑	9444.50	-40.23 %↓	18883.30	-40.23 %↓	28.56	-43.99 %↓
	FedRolex	0.68	6.14 %↑	9444.50	-40.23 %↓	18883.30	-40.23 %↓	28.56	-43.99 %↓
	Fjord	0.74	-2.47 %↓	9444.50	-40.23 %↓	18883.30	-40.23 %↓	28.56	-43.99 %↓
	HeteroFL	**0.80**	-10.11 %↓	9444.50	-40.23 %↓	18883.30	-40.23 %↓	28.56	-43.99 %↓
	SLT	0.78	-8.48 %↓	15482.69	-63.54 %↓	30955.33	-63.54 %↓	50.29	-68.19 %↓
CIFAR100	FedMBDP	0.39		**12341.86**		**24673.10**		**53.12**	
	FedAvg	0.48	-17.27 %↓	74990.08	-83.54 %↓	149931.65	-83.54 %↓	242.62	-78.11 %↓
	FD	0.34	16.69 %↑	18910.05	-34.73 %↓	37806.91	-34.74 %↓	65.98	-19.49 %↓
	FedRolex	0.38	5.12 %↑	18910.05	-34.73 %↓	37806.91	-34.74 %↓	65.98	-19.49 %↓
	Fjord	0.41	-4.87 %↓	18910.05	-34.73 %↓	37806.91	-34.74 %↓	65.98	-19.49 %↓
	HeteroFL	0.46	-13.73 %↓	18910.05	-34.73 %↓	37806.91	-34.74 %↓	65.98	-19.49 %↓
	SLT	**0.51**	-21.97 %↓	30712.93	-59.82 %↓	61403.69	-59.82 %↓	110.82	-52.07 %↓

TABLE IV
PERFORMANCE OF SIX ALGORITHMS ON DENSENET40

	Algorithm	Test Accuracy		Training Latency (s)		Computation (GFLOPs)		Communication (MB)	
CIFAR10	FedMBDP	0.73		**18408.45**		**36812.17**		**23.66**	
	FedAvg	0.77	-6.09 %↓	63288.82	-70.91 %↓	126564.22	-70.91 %↓	67.18	-64.79 %↓
	FD	0.61	18.62 %↑	31916.70	-42.32 %↓	63826.41	-42.32 %↓	34.94	-32.30 %↓
	FedRolex	0.63	16.02 %↑	31916.70	-42.32 %↓	63826.41	-42.32 %↓	34.94	-32.30 %↓
	Fjord	0.71	1.65 %↑	31916.70	-42.32 %↓	63826.41	-42.32 %↓	34.94	-32.30 %↓
	HeteroFL	**0.78**	-7.32 %↓	31916.70	-42.32 %↓	63826.41	-42.32 %↓	34.94	-32.30 %↓
CIFAR100	FedMBDP	0.37		**25701.77**		**51397.71**		**29.13**	
	FedAvg	0.40	-6.05 %↓	126599.47	-79.70 %↓	253170.23	-79.70 %↓	143.5	-79.70 %↓
	FD	0.19	95.17 %↑	31927.55	-19.50 %↓	63847.20	-19.50 %↓	39.54	-26.33 %↓
	FedRolex	0.20	90.78 %↑	31927.55	-19.50 %↓	63847.20	-19.50 %↓	39.54	-26.33 %↓
	Fjord	0.26	45.59 %↑	31927.55	-19.50 %↓	63847.20	-19.50 %↓	39.54	-26.33 %↓
	HeteroFL	**0.40**	-7.57 %↓	31927.55	-19.50 %↓	63847.20	-19.50 %↓	39.54	-26.33 %↓

FedMBDP's test accuracy of 0.72 for CIFAR-10 and 0.39 for CIFAR-100 is competitive but not the highest, it offers a practical balance between accuracy and training efficiency. Methods like SLT achieve higher test accuracies (0.78 for CIFAR-10 and 0.51 for CIFAR-100) but at the cost of much longer training times. The efficiency of FedMBDP makes it a compelling choice for scenarios where timely model updates are crucial, particularly in FL environments where training latency is a critical factor.

Table IV presents the performance of seven methods on DenseNet40. While FedMBDP does not achieve the highest test accuracy, it maintains the lowest training latency on both CIFAR-10 and CIFAR-100 datasets. On CIFAR-10, FedMBDP achieves a test accuracy of 0.73 and a training latency of 18408.45 seconds, which is significantly lower than most other algorithms. The dense structure of DenseNet40, which cannot be effectively divided into small blocks (only 3 blocks), impacts FedMBDP's performance. This limitation affects both test accuracy and training latency, as the dense connections hinder fine-grained division and parallel processing of FedMBDP.

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on October 27,2026 at 23:55:19 UTC from IEEE Xplore. Restrictions apply.

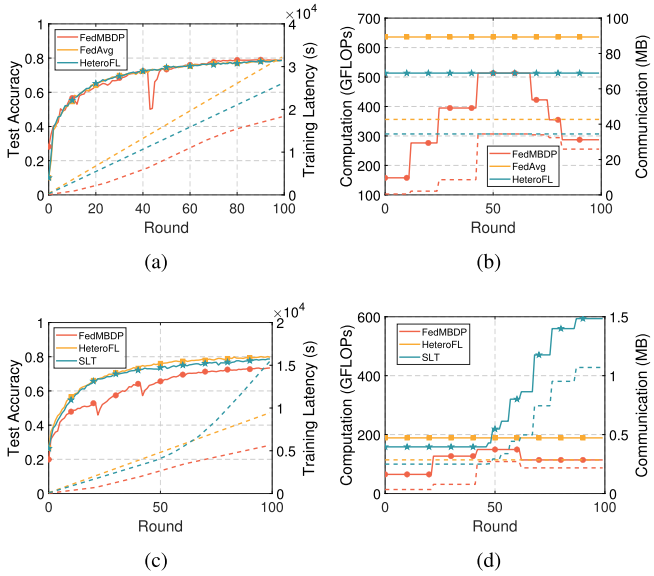


Fig. 5. Performance comparison on CIFAR10: (a) test accuracy and training latency with ResNet18; (b) computation and communication of each round with ResNet18; (c) test accuracy and training latency with ResNet20; (d) computation and communication of each round with ResNet20. The solid line corresponds to the left ordinate. The dashed line corresponds to the right ordinate.

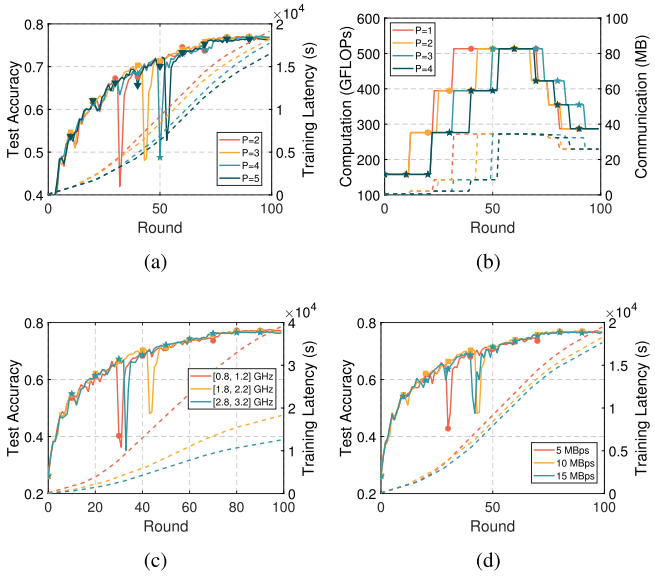


Fig. 6. Effects of hyper-parameters on CIFAR10 using ResNet18: (a) the effect of patience on test accuracy; (b) the effect of performance patience on training efficiency; (c) the effect of $1/C_n$ on test accuracy; (d) the effect of $1/R_n$ on test accuracy. The solid line corresponds to the left ordinate. The dashed line corresponds to the right ordinate.

D. Convergence Performance

Fig. 5 shows the performance of different methods on CIFAR-10 using ResNet18 and ResNet20. It can be observed that the test accuracy of the three methods is close. Notably, FedMBDP exhibits a sharp fluctuation in round 43 due to the inclusion of a new training block that needs to be trained. FedMBDP consistently shows the lowest training latency across all rounds, with its training latency being only half that of FedAvg in the final round. Examining the red dashed line, we notice three distinct stages: i) In the initial stage, training latency increases gradually. ii) In the middle stage, starting from round 40, training latency rises sharply. ii) In the final stage, training latency increases gradually again. The reason for these changes is evident in Fig. 5(b). As more training blocks participate, both computation and communication costs increase. However, in round 70, these costs decrease because some shallower blocks are frozen, reducing the overall computational burden.

It can be observed from Fig. 5(c) that FedMBDP performs worse than SLT and HeteroFL in terms of test accuracy. This is attributed to the fact that ResNet20 has only three blocks, limiting the ability of FedMBDP to progressively learn. Despite this, FedMBDP still achieves good test accuracy while dramatically reducing training latency. Fig. 5(d) shows that SLT requires significantly higher computation costs to achieve its better test accuracy. This highlights FedMBDP's efficiency in maintaining reasonable test accuracy with substantially lower computational demands.

E. Effects of Different Hyper-Parameters

From Fig. 6(a), the test accuracy (solid lines) of FedMBDP increases rapidly at first and then plateaus across all settings (P=2, P=3, P=4, P=5). We observe that the training latency (dashed lines) rises almost linearly with rounds, but higher values of P generally result in lower latency increases. This is because deeper blocks are added progressively later, delaying the incorporation of more complex layers. Fig. 6(b) confirms this trend. Overall, higher P values improve training efficiency by reducing latency and optimizing computation and communication demands.

Similarly, Fig. 6(c) and (d) show that higher computation frequency and communication rate also reduce training latency. This is due to more frequent updates and faster parameter transmission, improving overall training efficiency.

TABLE V
EFFECTS OF THRESHOLD S^* ON CIFAR10 USING RESNET18

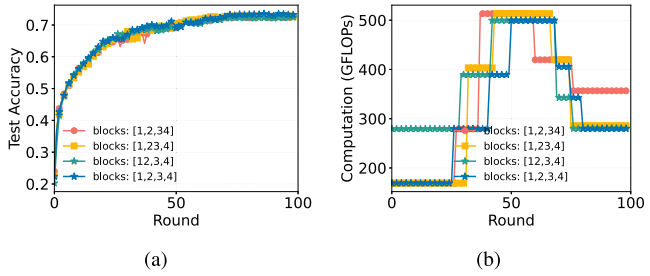
S^*	Test Accuracy	Training Latency (s)	Computation (GFLOPs)	Communication (MB)
7.5×10^{-3}	0.7275	19151.8948	37849.0418	2273.7396
8×10^{-3}	0.7275	19151.8948	37849.0418	2273.7396
8.5×10^{-3}	0.7238	18052.4650	35709.7343	1975.9783
9×10^{-3}	0.6861	19469.8486	38623.9098	1578.9373

Table V illustrates the impact of threshold S^* on the CIFAR-10 dataset using ResNet-18. The results demonstrate that S^* plays a critical role in balancing communication efficiency, computational workload, and model accuracy. As S^* increases from 7.5×10^{-3} to 9×10^{-3}, the communication overhead is significantly reduced by approximately 30.5% (from 2273.74MB to 1578.94MB) as fewer blocks meet the MBD criteria for global aggregation. Specifically, when S^* is set below 8×10^{-3}, the threshold is too conservative to filter out local updates, resulting in negligible impact on model performance. However, increasing S^* to 9×10^{-3} introduces aggressive filtering, causing the test accuracy to drop from 0.7275 to 0.6861 because critical model updates are omitted. This degradation directly impacts

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on October 27,2026 at 23:55:19 UTC from IEEE Xplore. Restrictions apply.

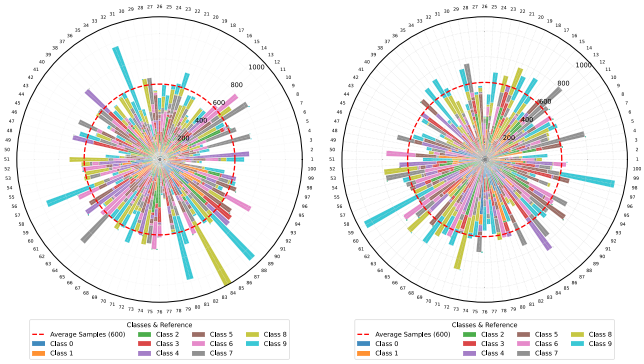
TABLE VI
PERFORMANCE OF 6 ALGORITHMS ON RESNET18 WITH CIFAR10 UNDER NON-IID SETTINGS. FOR NON-IID SETTINGS, WE OBEY THE DIRICHLET DISTRIBUTION TO SAMPLE DATA. $Dir(\alpha)$ DENOTES THE DIRICHLET DISTRIBUTION AND α IS A CONCENTRATION PARAMETER

	Algorithm	Test Accuracy		Training Latency (s)		Computation (GFLOPs)		Communication (MB)	
$\alpha = 0.5$	FedMBDP	**0.34**		**19544.23**		**38580.20**		**2541.27**	
	FedAvg	0.23	45.88 %↑	32220.54	-39.34 %↓	63587.26	-39.33 %↓	4269.13	-40.47 %↓
	FD	0.18	83.42 %↑	25230.09	-22.54 %↓	49784.49	-22.51 %↓	3378.43	-24.78 %↓
	FedRolex	0.19	79.21 %↑	25230.09	-22.54 %↓	49784.49	-22.51 %↓	3378.43	-24.78 %↓
	Fjord	0.19	79.21 %↑	25230.09	-22.54 %↓	49784.49	-22.51 %↓	3378.43	-24.78 %↓
	HeteroFL	0.22	53.12 %↑	25230.09	-22.54 %↓	49784.49	-22.51 %↓	3378.43	-24.78 %↓
$\alpha = 0.7$	FedMBDP	**0.38**		**19477.75**		**38470.32**		**2425.89**	
	FedAvg	0.35	7.89 %↑	32220.54	-39.55 %↓	63587.26	-39.50 %↓	4269.13	-43.18 %↓
	FD	0.19	98.04 %↑	25230.09	-22.80 %↓	49784.49	-22.73 %↓	3378.43	-28.19 %↓
	FedRolex	0.20	88.14 %↑	25230.09	-22.80 %↓	49784.49	-22.73 %↓	3378.43	-28.19 %↓
	Fjord	0.20	88.14 %↑	25230.09	-22.80 %↓	49784.49	-22.73 %↓	3378.43	-28.19 %↓
	HeteroFL	0.33	13.91 %↑	25230.09	-22.80 %↓	49784.49	-22.73 %↓	3378.43	-28.19 %↓



(a) (b)

Fig. 7. Effects of different combinations of blocks on CIFAR10 using ResNet18: (a) test accuracy; (b) computation (GFLOPs). We compare 4 different combinations about blocks, i.e., [1,2,3,4], [12,3,4], [1,23,4], [1,2,34].



(a) $\alpha = 0.7$ (b) $\alpha = 0.5$

Fig. 8. Visualizing Non-IID label distribution of CIFAR10 across edge servers.

the patience-based mechanism: lower accuracy triggers a more frequent reduction of the patience value P, thereby accelerating the transition between training blocks. Consequently, the resulting premature activation of deeper layers leads to variable training latencies and increased computational costs, as evidenced by the shift in computation from 37849.04 GFLOPs to 38623.91 GFLOPs. These findings highlight that while a larger S^* optimizes bandwidth, it must be carefully tuned to prevent accuracy loss from inadvertently increasing the computational burden through unstable block transitions.

F. Effects of Different Combinations of Blocks

To address the concerns regarding the generality of static partitioning, we conducted an extensive evaluation of different block combination strategies for ResNet-18, as illustrated in Fig. 7. The results indicate that the specific partitioning configuration has a pronounced impact on the initial training phase. Specifically, as shown in Fig. 7(a), configurations with fused blocks (e.g., [12, 3, 4] or [1, 2, 34]) exhibit different starting accuracies and convergence trajectories compared to the baseline 4-block partition [1, 2, 3, 4]. However, as the global aggregation rounds progress, all combinations eventually converge to a similar final test accuracy (approximately 0.72), suggesting that the layer-wise learning paradigm is robust to various structural groupings in the long term.

In contrast, the choice of block combination has a significant impact on computational overhead. As depicted in Fig. 7(b), the 4-block baseline maintains the lowest and most stable GFLOPs throughout the training process. In contrast, fused configuration, necessitated by the dense or residual connection properties of the architecture, tend to trigger deeper layer computations earlier or more frequently. This results in sharp fluctuations in computational demands, as evidenced by the peak shifts in GFLOPs for the [1, 2, 34] and [12, 3, 4] settings. These observations reinforce that while final accuracy is relatively invariant to the partitioning scheme, a fine-grained division that respects the model's natural structural boundaries is essential for minimizing computational latency in resource-constrained federated environments.

G. Effects of non-IID Data

Table VI summarizes the performance of six algorithms under Non-IID settings using the Dirichlet distribution ($\alpha = 0.5$ and $\alpha = 0.7$). As shown in Fig. 8, the absolute accuracy drop from the IID setting results from an extreme "dual-skew" environment. Combining severe label skew (only 2-3 classes per client) with massive data starvation inevitably triggers catastrophic local over-fitting across all evaluated methods when training ResNet18. However, the results demonstrate that FedMBDP consistently achieves superior test accuracy while maintaining significantly lower training latency compared to all state-of-the-art baselines. Specifically, at $\alpha = 0.5$, FedMBDP achieves

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on October 27,2026 at 23:55:19 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
PERFORMANCE OF DIFFERENT ABLATION SETTINGS. WE CONSIDER SEPARATELY REMOVING LOCAL TRAINING PROCESSING (LTP) AND GLOBAL AGGREGATION PROCESSING (GAP), WHERE 'W/O' REPRESENTS 'WITHOUT'

Algorithm	Test Accuracy	Training Latency (s)	Computation (GFLOPs)	Communication (MB)
FedMBDP	0.78	**19151.89**	**37849.04**	**2273.74**
FedMBDP w/o GAP	0.78	19351.43	**37849.04**	4269.13
FedMBDP w/o LTP	0.78	32087.63	63587.58	2973.74

a test accuracy of 0.34, representing a substantial improvement of 45.88% over FedAvg and up to 83.42% over FD. Critically, this performance gain is accompanied by a reduction in training latency of approximately 39.34% compared to the best-performing baseline.The simultaneous improvement in both accuracy and efficiency can be attributed to the synergy between our patience-based local training and MBD-based global aggregation. Unlike traditional methods that suffer from significant weight divergence under Non-IID data distributions, the block-wise training approach in FedMBDP limits the impact of noisy local updates by ensuring shallower blocks reach a stable convergence state before proceeding to deeper layers. Furthermore, MBD dynamically filters non-essential updates to simultaneously mitigate communication bottlenecks and naturally regularize the global model against divergent local features, preserving a robust, generalizable representation.

H. Ablation Study

We conduct ablation experiments regarding local training processing (LTP) and global aggregation processing (GAP) as shown in Table VII. According to the ablation results, both LTP and GAP are essential for reducing training costs. Although test accuracy remains constant at 0.78 across all configurations, removing LTP significantly increases training latency (from 19,151.89 seconds to 32,087.63 seconds) and computation (from 37,849.04 GFLOPs to 63,587.58 GFLOPs). Conversely, removing GAP dramatically increases communication costs (from 2,273.74 MB to 4,269.13 MB). These findings suggest that LTP is primarily responsible for reducing training time and computational load, while GAP effectively minimizes communication overhead. Consequently, FedMBDP proves to be an efficient approach in terms of both computation and communication.

I. Deployment in a Realistic Distributed Environment

To evaluate the practical efficacy of FedMBDP within a tangible distributed setting, we execute a physical deployment utilizing the enterprise-grade FEDn framework [43]. The experimental cluster comprises 5 heterogeneous devices: 2 high-performance servers connected to the internet via high-speed wired Ethernet (optical fiber) and 3 standard laptops connected via Wi-Fi. This configuration is designed to authentically emulate hardware heterogeneity and physical network constraints. We employ ResNet18 on CIFAR10 to assess system performance under these realistic edge-cloud conditions over 50 global communication rounds.

TABLE VIII
PERFORMANCE COMPARISON OF 6 ALGORITHMS ON RESNET18 WITH CIFAR10 DEPLOYED ON THE FEDN FRAMEWORK

Algorithm	Test Accuracy		Training Latency (s)	
FedMBDP	**0.72**		**5073.00**	
FedAvg	**0.72**	0.00	7518.00	-32.52 %↓
FD	0.69	4.35 %↑	6069.00	-16.41 %↓
FedRolex	0.62	16.13 %↑	6183.00	-17.95 %↓
Fjord	0.70	2.86 %↑	6205.00	-18.24 %↓
HeteroFL	**0.72**	0.00	6037.00	-15.97 %↓

The quantitative results, summarized in Table VIII, demonstrate that FedMBDP achieves a superior trade-off between model accuracy and training efficiency in a physical distributed environment. Notably, FedMBDP maintains a test accuracy of 0.72, matching the performance of the strongest baselines, specifically FedAvg and HeteroFL. Concurrently, it records the lowest total training latency of 5,073.00 seconds. This represents a substantial 32.52% reduction compared to FedAvg (7,518.00 s) and a reduction ranging from 16.41% to 26.37% compared to other state-of-the-art methods, including FD, FedRolex, and Fjord. These findings empirically verify that by decoupling the training process and optimizing block transmission, FedMBDP effectively mitigates the communication and computation bottlenecks inherent in real-world federated systems without compromising final model performance.

VII. CONCLUSIONS AND FUTURE WORK

This paper investigates the training efficiency of FL. We propose FedMBDP to address this problem through block-wise learning. FedMBDP decomposes the original problem into two key components: local training processing and global aggregation processing. We introduce a patience-based algorithm to adaptively determine the local training blocks and a mean block difference-based algorithm to dynamically select the uploading blocks. Extensive evaluations show that FedMBDP outperforms state-of-the-art baselines in terms of training latency across various neural network models.

FedMBDP performs effectively for neural network models with more blocks (e.g., ResNet18), reducing training latency and achieving better test accuracy. For highly coupled neural network models, FedMBDP shows limited effectiveness, where it reduces training latency but does not maintain test accuracy. In subsequent research, we plan to explore more adaptive partitioning strategies to address this limitation.

REFERENCES

[1] D. Chai, L. Wang, L. Yang, J. Zhang, K. Chen, and Q. Yang, "A survey for federated learning evaluations: Goals and measures," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 10, pp. 5007–5024, Oct. 2024.
[2] A. Z. Tan, H. Yu, L. Cui, and Q. Yang, "Towards personalized federated learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 12, pp. 9587–9603, Dec. 2023.
[3] Q. Li et al., "A survey on federated learning systems: Vision, hype and reality for data privacy and protection," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 4, pp. 3347–3366, Apr. 2023.
[4] P. Kairouz et al., "Advances and open problems in federated learning," *Foundations Trends Mach. Learn.*, vol. 14, no. 1-2, pp. 1–210, 2021.

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on October 27,2026 at 23:55:19 UTC from IEEE Xplore. Restrictions apply.

[5] S. Guo et al., "Byzantine-resilient decentralized stochastic gradient descent," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 32, no. 6, pp. 4096–4106, Jun. 2022.
[6] Y. Liu et al., "Vertical federated learning: Concepts, advances, and challenges," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 07, pp. 3615–3634, Jul. 2024.
[7] E. Hallaji, R. Razavi-Far, M. Saif, B. Wang, and Q. Yang, "Decentralized federated learning: A survey on security and privacy," *IEEE Trans. Big Data*, vol. 10, no. 2, pp. 194–213, Apr. 2024.
[8] N. Chen, T. Qiu, X. Zhou, S. Zhang, W. Si, and X. Wang, "LEGO-motif: Enhancing IoT topology robustness with evolutionary motif-based generation," *IEEE Trans. Syst., Man, Cybern.: Syst.*, vol. 56, no. 3, pp. 1630–1643, Mar. 2026.
[9] T. Wu et al., "A brief overview of ChatGPT: The history, status quo and potential future development," *IEEE/CAA J. Automatica Sinica*, vol. 10, no. 5, pp. 1122–1136, May 2023.
[10] M. Xu et al., "A survey of resource-efficient LLM and multimodal foundation models," 2024, *arXiv:2401.08092*.
[11] Y. Chen, Z. Chen, P. Wu, and H. Yu, "FedOBD: Opportunistic block dropout for efficiently training large-scale neural networks through federated learning," in *Proc. Int. Joint Conf. Artif. Intell.*, 2023, pp. 3541–3549.
[12] Z. Zhu et al., "FedLP: Layer-wise pruning mechanism for communication-computation efficient federated learning," in *Proc. IEEE Int. Conf. Commun.*, 2023, pp. 1250–1255.
[13] H. L. Xin'ao Wang, K. Chen, and L. Shou, "FEDBFPT: An efficient federated learning framework for BERT further pre-training," in *Proc. Int. Joint Conf. Artif. Intell.*, 2023, pp. 4344–4352.
[14] M. Kim, S. Yu, S. Kim, and S.-M. Moon, "DepthFL: Depthwise federated learning for heterogeneous clients," in *Proc. Int. Conf. Learn. Representations*, 2022, pp. 1–20.
[15] M. Jaderberg et al., "Decoupled neural interfaces using synthetic gradients," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 1627–1635.
[16] Z. Huo et al., "Decoupled parallel backpropagation with convergence guarantee," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 2098–2106.
[17] E. Belilovsky, M. Eickenberg, and E. Oyallon, "Decoupled greedy learning of CNNs," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 736–745.
[18] W. Ma, M. Yu, K. Li, and G. Wang, "Why layer-wise learning is hard to scale-up and a possible solution via accelerated downsampling," in *Proc. IEEE 32nd Int. Conf. Tools Artif. Intell.*, 2020, pp. 238–243.
[19] M. Raghu, J. Gilmer, J. Yosinski, and J. Sohl-Dickstein, "SVCCA: Singular vector canonical correlation analysis for deep learning dynamics and interpretability," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, vol. 30, pp. 6078–6087.
[20] Y. Chen, A. Yuille, and Z. Zhou, "Which layer is learning faster? A systematic exploration of layer-wise convergence rate for deep neural networks," in *Proc. Int. Conf. Learn. Representations*, 2022, pp. 1–19.
[21] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. Y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. Artif. Intell. Statist.*, 2017, pp. 1273–1282.
[22] A. Reisizadeh, A. Mokhtari, H. Hassani, A. Jadbabaie, and R. Pedarsani, "FedPAQ: A communication-efficient federated learning method with periodic averaging and quantization," in *Proc. Int. Conf. Artif. Intell. Statist.*, 2020, pp. 2021–2031.
[23] D. Wen, K.-J. Jeon, and K. Huang, "Federated dropout—A simple approach for enabling federated learning on resource constrained devices," *IEEE wireless Commun. Lett.*, vol. 11, no. 5, pp. 923–927, May 2022.
[24] E. Diao, J. Ding, and V. Tarokh, "HeteroFL: Computation and communication efficient federated learning for heterogeneous clients," in *Proc. 9th Int. Conf. Learn. Representations*, 2021, pp. 1–24.
[25] K. Pfeiffer, R. Khalili, and J. Henkel, "Aggregating capacity in FL through successive layer training for computationally-constrained devices," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 35386–35402.
[26] L. Qu, S. Song, and C.-Y. Tsui, "FedDQ: Communication-efficient federated learning with descending quantization," in *Proc. IEEE Glob. Commun. Conf.*, 2022, pp. 281–286.
[27] P. Han, S. Wang, and K. K. Leung, "Adaptive gradient sparsification for efficient federated learning: An online learning approach," in *Proc. Int. Conf. Distrib. Comput. Syst.*, 2020, pp. 300–310.
[28] M. M. Amiri, D. Gunduz, S. R. Kulkarni, and H. V. Poor, "Federated learning with quantized global model updates," 2020, *arXiv:2006.10672*.
[29] D. Rothchild et al., "FetchSGD: Communication-efficient federated learning with sketching," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 8253–8265.
[30] S. Horvath, S. Laskaridis, M. Almeida, I. Leontiadis, S. Venieris, and N. Lane, "FjORD: Fair and accurate federated learning under heterogeneous targets with ordered dropout," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 12876–12889.
[31] A. Li, J. Sun, P. Li, Y. Pu, H. Li, and Y. Chen, "Hermes: An efficient federated learning framework for heterogeneous mobile clients," in *Proc. 27th Annu. Int. Conf. Mobile Comput. Netw.*, 2021, pp. 420–437.
[32] H.-P. Wang, S. Stich, Y. He, and M. Fritz, "ProgFed: Effective, communication, and computation efficient federated learning by progressive training," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 23034–23054.
[33] Y. Bengio, P. Lamblin, D. Popovici, and H. Larochelle, "Greedy layer-wise training of deep networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2006, pp. 153–160.
[34] J. Chen et al., "Run, don't walk: Chasing higher flops for faster neural networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 12021–12031.
[35] Y. Liao, Y. Xu, H. Xu, Z. Yao, L. Wang, and C. Qiao, "Accelerating federated learning with data and model parallelism in edge computing," *IEEE/ACM Trans. Netw.*, vol. 32, no. 1, pp. 904–918, Feb. 2024.
[36] J. Konecn, H. B. McMahan, F. X. Yu, P. Richtárik, A. T. Suresh, and D. Bacon, "Federated learning: Strategies for improving communication efficiency," in Proc. Adv. Neural Inf. Process. Syst. Workshop, 2016, *pp. 1-10*.
[37] M. Bartlett and J. Cussens, "Integer linear programming for the Bayesian network structure learning problem," *Artif. Intell.*, vol. 244, pp. 258–271, 2017.
[38] K. Poularakis, J. Llorca, A. M. Tulino, I. Taylor, and L. Tassiulas, "Joint service placement and request routing in multi-cell mobile edge computing networks," in *Proc. IEEE Conf. Comput. Commun.*, 2019, pp. 10–18.
[39] W. Zhou, C. Xu, T. Ge, J. McAuley, K. Xu, and F. Wei, "BERT loses patience: Fast and robust inference with early exit," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 18330–18 341.
[40] C. He, S. Li, M. Soltanolkotabi, and S. Avestimehr, "PipeTransformer: Automated elastic pipelining for distributed training of Large-scale Models," in Proc. Int. Conf. Mach. Learn., 2021, *pp. 4150-4159*.
[41] S. Alam, L. Liu, M. Yan, and M. Zhang, "FedRolex: Model-heterogeneous federated learning with rolling sub-model extraction," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 29677–29 690.
[42] S. Caldas, J. Konečny, H. B. McMahan, and A. Talwalkar, "Expanding the reach of federated learning by reducing client resource requirements," 2018, *arXiv:1812.07210*.
[43] M. Ekmefjord et al., "Scalable federated machine learning with FEDn," in *Proc. 22nd IEEE Int. Symp. Cluster, Cloud Internet Comput.*, 2022, pp. 555–564.



Chuan Sun (Member, IEEE) received the BS degree from the Wuhan University of Science and Technology, Wuhan, China, in 2017, and the PhD degree from the School of Big Data and Software Engineering, Chongqing University, Chongqing, China, in 2023. From 2023 to 2025, He was a research fellow with the College of Computing and Data Science, Nanyang Technological University, Singapore. He is currently a lecturer with the School of Computer and Information Science, Chongqing Normal University, Chongqing. He has authored or coauthored more than 30 journal and conference papers in *IEEE Journal on Selected Areas in Communications*, *IEEE Transactions on Services Computing*, *IEEE Transactions on Knowledge and Data Engineering*, and ICPADS. His research interests include LLM compression, federated learning, and reinforcement learning. He was the recipient of the Runner-up of the 2024 Best Paper Award from IEEE Transactions on Services Computing by the IEEE Computer Society Publications Board.



Yuanyuan Chen (Member, IEEE) received the PhD degree from the College of Computing and Data Science, Nanyang Technological University, Singapore. He was a software engineer contributing to open-source artificial intelligence software stacks. He has authored or coauthored several papers in leading international conferences and journals. His research focuses on explainable and efficient federated learning. His work has been recognized with AAAI Innovative Applications of Artificial Intelligence awards.

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on October 27,2026 at 23:55:19 UTC from IEEE Xplore. Restrictions apply.



Xiaoli Tang is currently a research fellow with the College of Computing and Data Science, Nanyang Technological University, Singapore. Her research has been authored or coauthored in ICML, NeurIPS, KDD, IJCAI, *IEEE Transactions on Knowledge and Data Engineering*, ICME, CIKM, *IEEE Transactions on Neural Networks and Learning Systems*, *Neural Networks*, and *Knowledge-Based Systems*. Her research focuses on federated learning.



Han Yu (Senior Member, IEEE) received the PhD degree from the School of Computer Engineering, Nanyang Technological University (NTU), in 2014. From 2018 to 2024, he was a Nanyang assistant professor with the College of Computing and Data Science (CCDS) and NTU. He held the prestigious Lee Kuan Yew postdoctoral fellowship from 2015 to 2018. He is currently a tenured associate professor with CCDS and NTU, Singapore. He has authored or coauthored more than 300 research papers in leading international conferences and journals. He co-authored the first monograph on the topic of federated learning - Federated Learning. His research focuses on trustworthy federated learning. His research works have won multiple scientific awards. He is a distinguished member of CCF, and a senior member of AAAI.

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on October 27,2026 at 23:55:19 UTC from IEEE Xplore. Restrictions apply.